An Integrated Readable Companion to Zhao–Cui Tensor-Train Sequential Filtering and Fixed-Branch Gradients

BayesFilter P22 Technical Note

2026-06-02

Abstract

This P22 note is an integrated readable companion built from the P20 source-order reconstruction and the P21 implementation-orientation supplement. It does not summarize, shorten, or replace P20. Instead, it preserves P20's Zhao–Cui annotation and fixed-branch derivations, then inserts additional orientation, shape contracts, carried-filter storage details, and finite-difference reporting rules at the points where a reader or later implementer needs them. The document is intentionally generous: it favors explicit equations, dimensions, and implementation meaning over compactness.

Contents

1 Reader Contract, Non-Summary Rule, And Notation

This note uses "Zhao–Cui anchor" sentences to identify where a displayed formula comes from, but the anchor is not the explanation. Every important formula is restated in BayesFilter notation, derived from the previous formula, interpreted for filtering or transport, and tied to an implementation object.

The construction rule for P22 is

$$\text{P22} = \text{P20 mathematical spine} + \text{P21 reader orientation and implementation specification}, \tag{P22-C1}$$

not

$$\text{P22} = \text{a shorter summary of P20}. \tag{P22-C2}$$

The source-order Zhao–Cui annotation remains the spine through Sections 1–3 and 5. The added material has a narrower purpose: whenever an equation is difficult to read, it states what is being stored, what is being integrated, what is being differentiated, which branch choices are fixed, and what would break the mathematical claim.

Zhao–Cui write the unknown parameter as θ. The first part of this note follows that convention when reconstructing their paper. The BayesFilter derivative extension later writes the external differentiated parameter as β, because there θ would otherwise mean both a random coordinate and the argument of the likelihood function.

The state dimension is m, observation dimension is n, and parameter dimension is d. Thus

$$x_t = (x_{t,1}, \dots, x_{t,m}) \in \mathcal{X} \subseteq \mathbb{R}^m, \qquad y_t = (y_{t,1}, \dots, y_{t,n}) \in \mathcal{Y} \subseteq \mathbb{R}^n,$$

and

$$\theta = (\theta_1, \dots, \theta_d) \in \Theta \subseteq \mathbb{R}^d.$$

For coordinate groups, the paper uses

$$x_{t,<j} = (x_{t,1}, \dots, x_{t,j-1}), \qquad x_{t,>j} = (x_{t,j+1}, \dots, x_{t,m}), \tag{N1}$$

and similarly

$$x_{t,\le j} = (x_{t,1}, \dots, x_{t,j}), \qquad x_{t,\ge j} = (x_{t,j}, \dots, x_{t,m}). \tag{N2}$$

The point of this notation is conditional density bookkeeping. A lower triangular map conditions coordinate j on $x_{<j}$; an upper triangular map conditions it on $x_{>j}$.

All densities in the reconstruction are assumed absolutely continuous with respect to the relevant Lebesgue measure. Zhao–Cui use p for normalized posterior densities and π for unnormalized densities. Their approximations are $\widehat{p}$ and $\widehat{\pi}$. When p and $\widehat{p}$ are normalized densities, the Hellinger distance is

$$D_H(p,\widehat{p}) = \left[\frac{1}{2}\int \left(\sqrt{p(r)} - \sqrt{\widehat{p}(r)}\right)^2 \mathrm{d}r\right]^{1/2}. \tag{N3}$$

This distance appears because the squared-TT method approximates square roots of densities.

If $S : \mathcal{X} \to \mathcal{U}$ is a diffeomorphism and $X \sim p$, then the density of $U = S(X)$ is the pushforward

$$S_{\#}p(u) = p(S^{-1}(u))\left|\det \nabla S^{-1}(u)\right|. \tag{N4}$$

If $U \sim \eta$, then the density of $X = S^{-1}(U)$ is the pullback

$$S^{\#}\eta(x) = \eta(S(x))\left|\det \nabla S(x)\right|. \tag{N5}$$

These two identities are the entire mathematical engine behind the Knothe–Rosenblatt maps and the preconditioning maps in Section 5. The implementation stores an evaluator for S, an evaluator for S^{-1}, and the relevant log-Jacobian determinant when the map is used as a density transform.

2 Reader Orientation: Five Mathematical Objects

The Zhao–Cui method is easier to follow if the notation is organized around five objects. They are not five separate algorithms; they are five views of one sequential density approximation:

$$q_t, \qquad \phi_t, \qquad C_{t,k}, \qquad \widehat{Z}_t, \qquad \partial_\beta \log \widehat{Z}_t. \tag{P22-O1}$$

The first three describe the approximated density. The fourth is the scalar contributed to the likelihood. The fifth is the sensitivity needed by the BayesFilter fixed-branch extension.

Object 1: the unnormalized density

Bayes' rule begins with an unnormalized numerator:

$$\text{posterior density} = \frac{\text{likelihood} \times \text{prior}}{\int \text{likelihood} \times \text{prior}}. \tag{P22-O2}$$

In filtering, the prior at time t is the previous filter propagated through the transition model. A typical adjacent-state target has the form

$$q_t(x_t, x_{t-1}; \beta) = \widehat{p}_{t-1}(x_{t-1}; \beta) f_\beta(x_t \mid x_{t-1}) g_\beta(y_t \mid x_t). \tag{P22-O3}$$

The implementation question is more concrete than the notation suggests: which coordinates are used, and at which fitting points is q_t evaluated? A fixed branch stores

$$Z_{\text{fit}} = \begin{bmatrix} z_1^{(1)} & z_2^{(1)} \\ \vdots & \vdots \\ z_1^{(N)} & z_2^{(N)} \end{bmatrix}, \qquad \text{shape}(Z_{\text{fit}}) = (N, 2), \tag{P22-O4}$$

and a target vector

$$\widetilde{q}_t^{\text{fit}} = \left(\widetilde{q}_t(z^{(1)}; \beta), \ldots, \widetilde{q}_t(z^{(N)}; \beta)\right)^\top, \qquad \text{shape}(\widetilde{q}_t^{\text{fit}}) = (N,). \tag{P22-O5}$$

Object 2: the square-root approximation

A direct TT fit of q_t can be negative. Zhao–Cui therefore fit a square root and square it back:

$$\phi_t(z;\beta) \approx e^{c_t/2}\sqrt{\widetilde{q}_t(z;\beta)}, \tag{P22-O6}$$

$$\widehat{q}_t(z;\beta) = e^{-c_t}\phi_t(z;\beta)^2 + \tau_t\lambda_t(z). \tag{P22-O7}$$

The fitted values are

$$y_t^{\mathrm{fit}}[j] = e^{c_t/2}\sqrt{\widetilde{q}_t(z^{(j)};\beta)}, \qquad \mathrm{shape}(y_t^{\mathrm{fit}}) = (N,), \tag{P22-O8}$$

and the fixed-branch derivative values are

$$\dot{y}_t^{\mathrm{fit}}[j] = e^{c_t/2}\frac{\dot{\widetilde{q}}_t(z^{(j)};\beta)}{2\sqrt{\widetilde{q}_t(z^{(j)};\beta)}}. \tag{P22-O9}$$

Object 3: the tensor-train cores

For two coordinates, the fitted square root can be written

$$\phi_t(z_1,z_2;\beta) = \sum_{r=1}^{R}\left[\sum_{\ell=0}^{p-1} C_1[0,\ell,r]b_1(z_1)[\ell]\right]\left[\sum_{m=0}^{p-1} C_2[r,m,0]b_2(z_2)[m]\right]. \tag{P22-O10}$$

The stored arrays are

$$\mathrm{shape}(C_1) = \mathrm{shape}(\dot{C}_1) = (1,p,R), \qquad \mathrm{shape}(C_2) = \mathrm{shape}(\dot{C}_2) = (R,p,1). \tag{P22-O11}$$

The branch freezes the fitting rule, not the fitted numerical core values:

$$C_k(\beta;B) = \text{core returned by the fixed fitting equations at } \beta. \tag{P22-O12}$$

Object 4: the normalizer

The evidence increment is the mass of the declared approximate density:

$$\widehat{Z}_t(\beta) = e^{-c_t}R_t(\beta) + \tau_t, \qquad R_t(\beta) = \int \phi_t(z;\beta)^2\,\mathrm{d}z. \tag{P22-O13}$$

In TT form, this mass is computed by contractions rather than by a full tensor grid. For the two-coordinate case,

$$S_2[r,s] = \sum_{\ell,m} C_2[r,\ell,0]B_2[\ell,m]C_2[s,m,0], \tag{P22-O14}$$

$$R_t = \sum_{r,s,\ell,m} C_1[0,\ell,r]B_1[\ell,m]C_1[0,m,s]S_2[r,s]. \tag{P22-O15}$$

Object 5: the derivative of the log normalizer

The derivative used by the fixed-branch likelihood is

$$\partial_\beta \log \widehat{Z}_t = \frac{\dot{\widehat{Z}}_t}{\widehat{Z}_t}. \tag{P22-O16}$$

Since c_t, τ_t, λ_t are frozen in the fixed branch,

$$\dot{\widehat{Z}}_t = e^{-c_t} \dot{R}_t. \tag{P22-O17}$$

The derivative of the carried filter is equally important because it becomes part of the next target:

$$\dot{\widehat{p}}_t(z_t;\beta) = \frac{\dot{a}_t(z_t;\beta)}{\widehat{Z}_t(\beta)} - \frac{a_t(z_t;\beta)\dot{\widehat{Z}}_t(\beta)}{\widehat{Z}_t(\beta)^2}. \tag{P22-O18}$$

This front orientation is not a replacement for the derivation below. It is the small map used to read the longer source-order reconstruction.

3 What Problem Is Being Solved?

Source unit S1-overview. Zhao–Cui begin with a hidden Markov state, observations, and possibly unknown static parameters. The question answered here is: what density must a sequential algorithm update when new data arrive?

The data arrive in time order. At time t there is a hidden state

$$x_t \in \mathbb{R}^{m_x}$$

and an observation

$$y_t \in \mathbb{R}^{m_y}.$$

There may also be a static unknown parameter

$$\alpha \in \mathbb{R}^{m_\alpha}.$$

The filtering problem is to update the conditional density of the current state and parameter when y_t arrives:

$$p(x_t, \alpha \mid y_{1:t}).$$

The path problem is to update the density of the whole trajectory and parameter:

$$p(x_{0:t}, \alpha \mid y_{1:t}).$$

The smoothing problem asks for an earlier state after later data have been seen:

$$p(x_k \mid y_{1:t}), \qquad k < t.$$

The bottleneck is always an integral. At time $t-1$ suppose we know, or have approximated,

$$p(x_{t-1}, \alpha \mid y_{1:t-1}).$$

After seeing y_t, Bayes' rule introduces the unnormalized three-block density

$$q_t(x_t, \alpha, x_{t-1}) = p(x_{t-1}, \alpha \mid y_{1:t-1}) f_\alpha(x_t \mid x_{t-1}) g_\alpha(y_t \mid x_t).$$

The next filter is obtained by integrating out x_{t-1}:

$$p(x_t, \alpha \mid y_{1:t}) = \frac{\int q_t(x_t, \alpha, x_{t-1})\,\mathrm{d}x_{t-1}}{\int q_t(x_t, \alpha, x_{t-1})\,\mathrm{d}x_t\,\mathrm{d}\alpha\,\mathrm{d}x_{t-1}}.$$

In low dimension one can attack this integral by quadrature, particles, or Gaussian projection. Zhao–Cui's idea is different: approximate the whole function q_t in a separable tensor-train form so that marginal integrals are cheap contractions instead of high-dimensional numerical integrations.

Implementation object. A minimal implementation must expose evaluators for $\widehat{p}_{t-1}$, f_α, and g_α, then build an evaluator for q_t. The first failure check is that q_t must be finite and nonnegative at all fitting points; otherwise neither a square root nor an importance weight is meaningful.

4 State-Space Model And BayesFilter Notation

Source units S1-Eq1–Eq2. The first two source equations answer: what conditional independence structure defines the state-space model? The variables $X_t \in \mathcal{X} \subseteq \mathbb{R}^m$ are latent states, $Y_t \in \mathcal{Y} \subseteq \mathbb{R}^n$ are observations, and $\alpha \in \Theta \subseteq \mathbb{R}^d$ is the static parameter.

Zhao–Cui use θ for the unknown parameter. In this note the parameter is written α, because θ is often used later as a generic argument in derivative formulas. The paper's model equations (1)–(2) become

$$X_t \mid (X_{0:t-1} = x_{0:t-1}, \alpha) \equiv X_t \mid (X_{t-1} = x_{t-1}, \alpha) \sim f_\alpha(x_t \mid x_{t-1}), \tag{BF-1}$$

and

$$Y_t \mid (X_{0:t} = x_{0:t}, Y_{1:t-1} = y_{1:t-1}, \alpha) \equiv Y_t \mid (X_t = x_t, \alpha) \sim g_\alpha(y_t \mid x_t). \tag{BF-2}$$

The first statement says the latent state process is Markov once α is fixed. The second says the new observation only sees the current state, again once α is fixed.

Implementation contract for S1-Eq1–Eq2. Inputs are (x_t, x_{t-1}, α) for the transition evaluator and (y_t, x_t, α) for the observation evaluator. Outputs are nonnegative densities or log densities. Persisted state is the model specification and the observed y_t. Diagnostics are finite log-density checks and shape checks for m, n, d.

Source unit S1-Eq3. The next question is: how do the local transition and observation densities combine into the full joint density?

The joint density of parameter, states, and observations through time t, corresponding to Zhao–Cui Eq. (3), is

$$p(\alpha, x_{0:t}, y_{1:t}) = p(\alpha)p_\alpha(x_0)\prod_{j=1}^{t} f_\alpha(x_j \mid x_{j-1})g_\alpha(y_j \mid x_j). \tag{BF-3}$$

This is obtained by the chain rule:

$$\begin{aligned} p(\alpha, x_{0:t}, y_{1:t}) &= p(\alpha)p_\alpha(x_0)\prod_{j=1}^{t} p(x_j \mid x_{0:j-1}, y_{1:j-1}, \alpha)p(y_j \mid x_{0:j}, y_{1:j-1}, \alpha) \\ &= p(\alpha)p_\alpha(x_0)\prod_{j=1}^{t} f_\alpha(x_j \mid x_{j-1})g_\alpha(y_j \mid x_j), \end{aligned}$$

where the last equality uses the Markov and observation assumptions.

The derivation is the probabilistic analogue of multiplying conditional reaction probabilities along a time-ordered chain. Every new time index adds one transition factor and one observation factor. An implementation stores this as a log-density sum, not as a product, to avoid underflow:

$$\log p(\alpha, x_{0:t}, y_{1:t}) = \log p(\alpha) + \log p_\alpha(x_0) + \sum_{j=1}^{t} \{\log f_\alpha(x_j \mid x_{j-1}) + \log g_\alpha(y_j \mid x_j)\}. \tag{BF-3a}$$

The diagnostic is simple: any impossible model transition should produce $-\infty$ log density intentionally, not a NaN.

Source unit S1-Eq4. The fourth source equation answers: how does the joint density become a posterior density after data are observed?

Conditioning on data gives Zhao–Cui Eq. (4) in BayesFilter notation:

$$p(\alpha, x_{0:t} \mid y_{1:t}) = \frac{p(\alpha, x_{0:t}, y_{1:t})}{p(y_{1:t})}, \qquad p(y_{1:t}) = \int p(\alpha, x_{0:t}, y_{1:t}) \, \mathrm{d}\alpha \, \mathrm{d}x_{0:t}. \tag{BF-4}$$

The denominator is the evidence. It is a scalar, but it is usually not available analytically because the integral spans the parameter and all states.

Implementation contract for S1-Eq4. The numerator is evaluable pointwise. The evidence is a scalar integral. A filtering algorithm that claims a likelihood must specify how it computes or approximates this scalar. The failure check is that the approximated evidence must be positive and finite.

5 The Four Marginal Learning Problems

Source units S1-Eq5–Eq8. The next four source equations answer: which posterior marginals correspond to filtering, parameter learning, path estimation, and smoothing?

Zhao–Cui Eqs. (5)–(8) are all marginalizations of the same posterior $p(\alpha, x_{0:t} \mid y_{1:t})$. Filtering keeps only the current state:

$$p(x_t \mid y_{1:t}) = \int p(\alpha, x_{0:t} \mid y_{1:t}) \, \mathrm{d}x_{0:t-1} \, \mathrm{d}\alpha. \tag{BF-5}$$

Parameter learning keeps only α:

$$p(\alpha \mid y_{1:t}) = \int p(\alpha, x_{0:t} \mid y_{1:t}) \, \mathrm{d}x_{0:t}. \tag{BF-6}$$

Path learning keeps the full state path but removes α:

$$p(x_{0:t} \mid y_{1:t}) = \int p(\alpha, x_{0:t} \mid y_{1:t}) \, \mathrm{d}\alpha. \tag{BF-7}$$

Fixed-lag or retrospective smoothing keeps one old state:

$$p(x_k \mid y_{1:t}) = \int p(\alpha, x_{0:t} \mid y_{1:t}) \, \mathrm{d}x_{0:k-1} \, \mathrm{d}x_{k+1:t} \, \mathrm{d}\alpha, \qquad k < t. \tag{BF-8}$$

The important point is not the names. It is that the same high-dimensional posterior density produces all four outputs by integration. A method that stores only a cloud of samples can estimate

these integrals by sample averages. A method that stores a Gaussian can compute only Gaussian marginals. A method that stores a tensor-train density aims to keep a non-Gaussian function representation while making many marginal integrals cheap.

Implementation contract for S1-Eq5–Eq8. The stored object must support integrating out named coordinate blocks. The outputs are marginal density evaluators. The failure mode is inconsistent normalization: if integrating a marginal over its retained coordinates does not give one, the contraction or normalizer is wrong.

6 The Exact Recursive Bottleneck

Source units S2.1-Eq9–Eq11. These source equations answer: how can the posterior update be performed sequentially without storing the whole old path forever?

We now derive Zhao–Cui Eqs. (9)–(11). Start from Bayes' rule:

$$p(\alpha, x_{0:t} \mid y_{1:t}) = \frac{p(\alpha, x_{0:t}, y_{1:t})}{p(y_{1:t})}.$$

Using the factorization (BF-3),

$$p(\alpha, x_{0:t}, y_{1:t}) = p(\alpha, x_{0:t-1}, y_{1:t-1}) f_\alpha(x_t \mid x_{t-1}) g_\alpha(y_t \mid x_t).$$

Also,

$$p(\alpha, x_{0:t-1}, y_{1:t-1}) = p(\alpha, x_{0:t-1} \mid y_{1:t-1}) p(y_{1:t-1}).$$

Therefore

$$\begin{aligned} p(\alpha, x_{0:t} \mid y_{1:t}) &= \frac{p(\alpha, x_{0:t-1} \mid y_{1:t-1}) p(y_{1:t-1}) f_\alpha(x_t \mid x_{t-1}) g_\alpha(y_t \mid x_t)}{p(y_{1:t})} \\ &= \frac{p(\alpha, x_{0:t-1} \mid y_{1:t-1}) f_\alpha(x_t \mid x_{t-1}) g_\alpha(y_t \mid x_t)}{p(y_t \mid y_{1:t-1})}. \end{aligned} \tag{BF-9}$$

The last step uses $p(y_{1:t}) = p(y_t \mid y_{1:t-1}) p(y_{1:t-1})$.

Now integrate (BF-9) over $x_{0:t-2}$. The only factor depending on the old path $x_{0:t-2}$ is $p(\alpha, x_{0:t-1} \mid y_{1:t-1})$, so

$$\begin{aligned} p(\alpha, x_t, x_{t-1} \mid y_{1:t}) &= \int p(\alpha, x_{0:t} \mid y_{1:t}) \, \mathrm{d}x_{0:t-2} \\ &= \frac{p(\alpha, x_{t-1} \mid y_{1:t-1}) f_\alpha(x_t \mid x_{t-1}) g_\alpha(y_t \mid x_t)}{p(y_t \mid y_{1:t-1})}. \end{aligned} \tag{BF-10}$$

Finally, integrate over x_{t-1} to obtain the next parameter-state filter:

$$p(\alpha, x_t \mid y_{1:t}) = \int p(\alpha, x_t, x_{t-1} \mid y_{1:t}) \, \mathrm{d}x_{t-1}. \tag{BF-11}$$

The numerator in (BF-10) is pointwise evaluable if the previous filter is evaluable. The denominator and the marginalization in (BF-11) are the hard parts. Zhao–Cui turn this into a function-approximation problem by replacing the unnormalized numerator with a tensor train.

Implementation contract for S2.1-Eq9–Eq11. Inputs are the retained evaluator for $p(\alpha, x_{t-1} \mid y_{1:t-1})$, the new observation y_t, and model evaluators f_α, g_α. The output of the numerator step is a pointwise evaluator for the three-block target. The persisted state after marginalization is an evaluator for $p(\alpha, x_t \mid y_{1:t})$ and an evidence increment. The failure check is that the marginal integral over (α, x_t) must equal one after normalization.

7 Tensor Trains From First Principles

Source units S2.2-TT-display and basis expansion. The question answered here is: what separable function class will replace a full high-dimensional grid?

This section starts from the object a numerical reader can picture: a table of function values. If $D = 2$, a function $h(r_1, r_2)$ sampled on a grid is a matrix. Low-rank matrix approximation writes

$$h(r_1, r_2) \approx \sum_{\rho=1}^{R} u_\rho(r_1) v_\rho(r_2).$$

Each term is separable: one factor depends only on r_1, one factor only on r_2. A tensor train is the many-variable analogue. It does not require a full D-dimensional table. It stores a chain of low-rank links, so that information can pass from coordinate 1 to coordinate D through the rank indices.

Let $r = (r_1, \dots, r_D) \in \Omega_1 \times \cdots \times \Omega_D$. A functional tensor train approximates a scalar function $h(r)$ by multiplying matrix-valued one-dimensional functions:

$$h(r) \approx \widehat{h}(r) = H_1(r_1) H_2(r_2) \cdots H_D(r_D), \tag{TT-1}$$

where

$$H_k(r_k) \in \mathbb{R}^{R_{k-1} \times R_k}, \qquad R_0 = R_D = 1.$$

Zhao–Cui's displayed TT formula includes the endpoint rank indices:

$$\widehat{h}(r) = \sum_{\rho_0=1}^{R_0} \sum_{\rho_1=1}^{R_1} \cdots \sum_{\rho_D=1}^{R_D} H_1^{(\rho_0,\rho_1)}(r_1) \cdots H_D^{(\rho_{D-1},\rho_D)}(r_D), \qquad R_0 = R_D = 1. \tag{TT-1a}$$

The endpoint sums have only one term. They are written so every core has the same two rank labels. A programmer usually evaluates the matrix product (TT-1); a proof often expands the rank contractions as in (TT-1a).

Because the first object has one row and the last has one column, the product is a 1×1 scalar. Written with indices,

$$\widehat{h}(r) = \sum_{\rho_1=1}^{R_1} \cdots \sum_{\rho_{D-1}=1}^{R_{D-1}} H_1^{1,\rho_1}(r_1) H_2^{\rho_1,\rho_2}(r_2) \cdots H_D^{\rho_{D-1},1}(r_D). \tag{TT-2}$$

The numbers R_k are TT ranks. A high rank lets the representation express strong interactions across a split $(r_1, \dots, r_k)$ and $(r_{k+1}, \dots, r_D)$. A low rank is useful only when the function has low-dimensional structure across that split.

To see the role of the ranks, split the variables into a left block and a right block:

$$r_{\le k} = (r_1, \dots, r_k), \qquad r_{>k} = (r_{k+1}, \dots, r_D).$$

Equation (TT-2) can be regrouped as

$$\widehat{h}(r_{\le k}, r_{>k}) = \sum_{\rho=1}^{R_k} L_\rho(r_{\le k}) R_\rho(r_{>k}), \tag{TT-2a}$$

where

$$L_\rho(r_{\le k}) = \sum_{\rho_1, \dots, \rho_{k-1}} H_1^{1,\rho_1}(r_1) \cdots H_k^{\rho_{k-1},\rho}(r_k),$$

and

$$R_\rho(r_{>k}) = \sum_{\rho_{k+1},\dots,\rho_{D-1}} H_{k+1}^{\rho,\rho_{k+1}}(r_{k+1}) \cdots H_D^{\rho_{D-1},1}(r_D).$$

Thus R_k is the number of separated terms allowed across that split. If a posterior density has only local or weak long-range dependence in the chosen ordering, moderate ranks can be enough. If a posterior has strong global coupling, the ranks grow and the method becomes expensive.

The low-dimensional analogy is the singular value decomposition of a matrix. For a two-coordinate function, rank R means R separable columns times rows. For many coordinates, R_k plays the same role at the cut after coordinate k. The failure diagnostic is rank growth: if any fitted R_k approaches the imposed maximum, the chosen coordinate ordering, basis, or preconditioner is not expressing the density compactly.

Each univariate matrix entry is represented in a basis:

$$H_k^{a,b}(r_k) = \sum_{\ell=0}^{p_k-1} C_k[a,\ell,b]\psi_{k,\ell}(r_k). \tag{TT-3}$$

This is the paper's basis expansion with renamed coefficient tensor. Zhao–Cui write the coefficient as $A_k[\alpha_{k-1}, j, \alpha_k]$; here it is $C_k[a,\ell,b]$. The three axes are left rank, basis index, and right rank. The basis index describes one-coordinate shape; the rank indices transmit dependence to neighboring coordinates.

The tensor $C_k \in \mathbb{R}^{R_{k-1}\times p_k \times R_k}$ is the k-th TT core. Thus an implementation stores the basis family, the domain map for each coordinate, and the core arrays $C_1, \dots, C_D$. To evaluate the function at one point r, it evaluates all basis vectors

$$\psi_k(r_k) = (\psi_{k,0}(r_k), \dots, \psi_{k,p_k-1}(r_k))^\top,$$

forms the matrices $H_k(r_k)$, and multiplies them.

Implementation contract for S2.2. Inputs are a coordinate r_k, basis metadata, and core tensor C_k. The operation is basis evaluation followed by a contraction over the basis index:

$$H_k(r_k) = \sum_{\ell=0}^{p_k-1} \psi_{k,\ell}(r_k)\, C_k[:,\ell,:]. \tag{TT-3a}$$

The persisted state is the list of cores, basis objects, and coordinate domains. The failure checks are basis overflow, invalid coordinate outside the domain, and incompatible adjacent ranks.

7.1 A Concrete Basis Choice

Source unit S2.2-basis-family. Zhao–Cui allow several basis families. This note fixes one basis to make the paper's abstract core formula executable.

Zhao–Cui's software supports several basis families. For a self-contained minimal implementation, choose one. On a finite interval $[a_k, b_k]$, map a physical coordinate r_k to

$$z_k = \frac{2r_k - (a_k + b_k)}{b_k - a_k} \in [-1, 1], \qquad r_k = \frac{a_k + b_k}{2} + \frac{b_k - a_k}{2} z_k. \tag{TT-6}$$

Use normalized Legendre polynomials

$$\psi_{k,\ell}(r_k) = \sqrt{\frac{2\ell+1}{b_k - a_k}}\, P_\ell(z_k), \qquad \ell = 0, \dots, p_k - 1. \tag{TT-7}$$

The Legendre polynomials are evaluated by the recurrence

$$P_0(z) = 1, \qquad P_1(z) = z,$$

$$(\ell+1)P_{\ell+1}(z) = (2\ell+1)zP_\ell(z) - \ell P_{\ell-1}(z), \qquad \ell \geq 1. \tag{TT-8}$$

This basis is useful pedagogically because its mass matrix is the identity:

$$\int_{a_k}^{b_k} \psi_{k,\ell}(r_k)\psi_{k,\ell'}(r_k)\,\mathrm{d}r_k = \delta_{\ell\ell'}. \tag{TT-9}$$

If another basis is used, every formula below remains the same after replacing the identity mass matrix by the corresponding one-dimensional mass matrix.

Zhao–Cui state that functional alternating schemes with cross interpolation typically use

$$O(DpR^2) \quad \text{target evaluations} \tag{TT-10}$$

and

$$O(DpR^3) \quad \text{floating point operations,} \tag{TT-11}$$

where $p = \max_k p_k$ and $R = \max_k R_k$. This is a cost statement after the ranks are moderate; it is not a promise that R stays small. The implementation should therefore record fitted ranks, residuals, and ill-conditioning diagnostics.

8 Why TT Marginalization Is Cheap Once The Representation Exists

Source unit S2.2-core-integration. The question answered here is: once a function is in TT form, how does the algorithm integrate out one coordinate without returning to a full grid?

Suppose h is represented by (TT-1). If we need to integrate out coordinate r_k, the product structure gives

$$\int \widehat{h}(r_1,\dots,r_D)\,\mathrm{d}r_k = H_1(r_1)\cdots H_{k-1}(r_{k-1})\left(\int H_k(r_k)\,\mathrm{d}r_k\right)H_{k+1}(r_{k+1})\cdots H_D(r_D). \tag{TT-4}$$

Only the k-th core changes. Its integrated matrix is

$$\overline{H}_k = \int H_k(r_k)\,\mathrm{d}r_k, \qquad \overline{H}_k[a,b] = \sum_{\ell=0}^{p_k-1} C_k[a,\ell,b]\int \psi_{k,\ell}(r_k)\,\mathrm{d}r_k. \tag{TT-5}$$

If the one-dimensional basis integrals are precomputed, marginalizing one coordinate is a small matrix contraction. Repeatedly applying (TT-4) integrates whole blocks of variables.

This explains why Zhao–Cui target a TT representation of (x_t, α, x_{t-1}). Once that three-block function is separable enough, the new filter $p(x_t, \alpha \mid y_{1:t})$ is obtained by integrating the old-state block x_{t-1}, one coordinate at a time.

Implementation contract for S2.2-core-integration. Inputs are the TT cores and precomputed one-dimensional basis integrals. The operation replaces one core by $\overline{H}_k$ and contracts adjacent matrices. The persisted object is a lower-dimensional TT or evaluator. The failure check is dimension/order consistency: integrating the wrong block silently returns the wrong filter.

9 Zhao–Cui Algorithm 1, Fully Annotated

Source unit S2.3-Algorithm1(a). Algorithm 1(a) answers: what target can be evaluated at time t using only the retained approximation from time $t-1$?

The exact recursion (BF-10) contains the previous exact filter. A numerical algorithm has the previous approximation. Let $\widehat{\pi}_{t-1}(x_{t-1}, \alpha)$ denote an unnormalized approximation of $p(x_{t-1}, \alpha \mid y_{1:t-1})$. Zhao–Cui Eq. (12) becomes

$$q_t(x_t, \alpha, x_{t-1}) = \widehat{\pi}_{t-1}(x_{t-1}, \alpha) f_\alpha(x_t \mid x_{t-1}) g_\alpha(y_t \mid x_t). \tag{A1-1}$$

This is not separable in general. Even if $\widehat{\pi}_{t-1}$ is a TT over (α, x_{t-1}), the transition and likelihood couple the new and old state coordinates.

Zhao–Cui use approximate proportionality before normalization. In explicit terms, $h_1 \propto_{\text{approx}} h_2$ means

$$\frac{h_1(r)}{\int h_1(s)\,\mathrm{d}s} \approx \frac{h_2(r)}{\int h_2(s)\,\mathrm{d}s}. \tag{A1-0}$$

This is why the algorithm can store an unnormalized TT. The normalization is not forgotten; it is postponed until the complete TT contraction $\widehat{Z}_t$ is computed.

Algorithm 1 has three mathematical steps.

Step 1: build a pointwise target. Given $r_t = (x_t, \alpha, x_{t-1})$, evaluate $q_t(r_t)$ by multiplying the previous approximation, transition density, and likelihood:

$$r_t \mapsto \widehat{\pi}_{t-1}(x_{t-1}, \alpha) f_\alpha(x_t \mid x_{t-1}) g_\alpha(y_t \mid x_t).$$

Storage needed: an evaluator for the previous TT approximation and evaluators for f_α and g_α.

Mini implementation contract. Inputs: $\widehat{\pi}_{t-1}$, f_α, g_α, y_t, and a point (x_t, α, x_{t-1}). Output: q_t or $\log q_t$. Persisted state: none beyond the evaluator. Operation: multiply three factors or add three log factors. Diagnostic: reject NaN, negative density values, or inconsistent coordinate block sizes.

Step 2: re-approximate by a TT.

Source unit S2.3-Algorithm1(b). This source unit answers: how does the nonseparable target become a separable object that can be marginalized?

Choose a variable order, in the paper usually

$$(x_t, \alpha, x_{t-1}).$$

Fit a TT

$$\widehat{\pi}_t(x_t, \alpha, x_{t-1}) = F_{t,1}(x_{t,1}) \cdots F_{t,m_x}(x_{t,m_x}) A_{t,1}(\alpha_1) \cdots A_{t,m_\alpha}(\alpha_{m_\alpha}) H_{t,1}(x_{t-1,1}) \cdots H_{t,m_x}(x_{t-1,m_x}) \tag{A1-2}$$

so that $\widehat{\pi}_t \approx q_t$. Zhao–Cui use functional TT tools with alternating approximation and rank adaptation. For implementation one must choose a domain map, basis, fitting points, rank rule, and stopping rule. The paper's research code supplies one such family; the fixed-branch section below supplies a differentiable BayesFilter specialization.

The most direct implementation view is least squares. Choose fitting points

$$r^{(j)} = (x_t^{(j)}, \alpha^{(j)}, x_{t-1}^{(j)}), \qquad j = 1, \ldots, N_{\text{fit}},$$

and define target values

$$y_j = q_t(r^{(j)}).$$

For fixed TT ranks and fixed all cores except core k, the fitted function is linear in the entries of core C_k. There is a design row $A_{j,k}$ such that

$$\widehat{\pi}_t(r^{(j)}) = A_{j,k} g_k, \qquad g_k = \mathrm{vec}(C_k). \tag{A1-2a}$$

Let $L_{j,k}$ be the left environment obtained by multiplying all cores before k at point $r^{(j)}$, and let $R_{j,k}$ be the right environment from all cores after k. Let $\psi_k(r_k^{(j)})$ be the local basis vector. Then

$$A_{j,k} = R_{j,k}^\top \otimes \psi_k(r_k^{(j)})^\top \otimes L_{j,k}. \tag{A1-2b}$$

This formula is worth lingering over. The left environment tells the current core what the earlier coordinates have already contributed. The right environment tells it what later coordinates will contribute. The basis vector gives the local coordinate dependence. The Kronecker product stitches these three pieces into a row that multiplies the vectorized core.

With weights $w_j > 0$, the core update solves

$$\min_{g_k} \sum_{j=1}^{N_{\mathrm{fit}}} w_j \big(A_{j,k} g_k - y_j\big)^2 + \rho \|g_k\|_2^2, \tag{A1-2c}$$

or equivalently

$$(A_k^\top W A_k + \rho I) g_k = A_k^\top W y. \tag{A1-2d}$$

An alternating scheme sweeps through the cores, updating one core at a time. Zhao–Cui's adaptive implementation is more sophisticated, but this least- squares view is the easiest way to see how a TT approximation becomes an actual numerical object.

Mini implementation contract. Inputs: target evaluator q_t, domains, basis objects, fitting points, weights, ranks, and sweep settings. Output: cores F, A, H for the fitted TT. Persisted state: cores, rank vector, basis metadata, fit residual, and any scaling shift. Operation: alternating core least-squares or cross approximation. Diagnostic: weighted residual, condition number of each normal matrix, rank growth, and negative values if this basic non-squared algorithm is interpreted as a density.

Step 3: integrate the old state.

Source unit S2.3-Algorithm1(c). This source unit answers: after the three-block target is fitted, what is retained for the next sequential update?

The next approximate filter is

$$\widehat{\pi}_t(x_t, \alpha) = \int \widehat{\pi}_t(x_t, \alpha, x_{t-1})\, \mathrm{d}x_{t-1}. \tag{A1-3}$$

Because x_{t-1} occupies the last block, this integral is a sequence of core integrals:

$$\widehat{\pi}_t(x_t, \alpha) = F_{t,1}(x_{t,1}) \cdots F_{t,m_x}(x_{t,m_x}) A_{t,1}(\alpha_1) \cdots A_{t,m_\alpha}(\alpha_{m_\alpha}) \overline{H}_{t,1} \cdots \overline{H}_{t,m_x}. \tag{A1-4}$$

The approximate normalizer is

$$\widehat{Z}_t = \int \widehat{\pi}_t(x_t, \alpha, x_{t-1})\, \mathrm{d}x_t\ \mathrm{d}\alpha\ \mathrm{d}x_{t-1}, \tag{A1-5}$$

again a complete TT contraction.

The same fitted TT gives the parameter marginal and filtering marginal:

$$\widehat{p}_t(\alpha) = \frac{\int \widehat{\pi}_t(x_t, \alpha, x_{t-1})\,\mathrm{d}x_t\;\mathrm{d}x_{t-1}}{\widehat{Z}_t}, \qquad \widehat{p}_t(x_t) = \frac{\int \widehat{\pi}_t(x_t, \alpha, x_{t-1})\,\mathrm{d}\alpha\;\mathrm{d}x_{t-1}}{\widehat{Z}_t}. \tag{A1-6}$$

The retained object for the next time step is not merely a scalar normalizer. It is the evaluable two-block density

$$\widehat{\pi}_t(x_t, \alpha) = \int \widehat{\pi}_t(x_t, \alpha, x_{t-1})\,\mathrm{d}x_{t-1}, \tag{A1-7}$$

because time $t+1$ will plug this function into the next target q_{t+1}.

Mini implementation contract. Inputs: fitted three-block TT and block ordering (x_t, α, x_{t-1}). Outputs: retained evaluator $\widehat{\pi}_t(x_t, \alpha)$, normalizer $\widehat{Z}_t$, optional marginals $\widehat{p}_t(x_t)$ and $\widehat{p}_t(\alpha)$. Persisted state: retained TT/evaluator and contraction metadata. Operation: endpoint TT core integration. Diagnostic: $\widehat{Z}_t$ must be finite and positive, and normalizing retained marginals must integrate to one.

The main failure mode of Algorithm 1 is sign. A fitted TT is a real-valued function approximation. Truncation or interpolation can make $\widehat{\pi}_t(r_t) < 0$, which is impossible for a density. Negative values also break transport maps and importance-sampling ratios. This motivates the squared-TT construction.

10 Why Nonnegativity Fails And Why Square-Root TT Repairs It

Source unit S3.1-nonnegativity-barrier. This source unit answers: why is Algorithm 1 not enough for transport maps and importance correction?

Approximating a nonnegative function by a truncated expansion does not preserve nonnegativity. The one-dimensional analogy is simple:

$$h(x) \geq 0 \quad \text{but} \quad \widehat{h}(x) = \sum_{\ell=0}^{p-1} c_\ell \psi_\ell(x) \text{ may be negative.}$$

The tensor-train case has the same issue. A low-rank approximation can reduce error in a least-squares sense while still creating small negative regions.

Zhao–Cui's repair is to approximate the square root of an unnormalized density. If

$$\sqrt{\pi(r)} \approx \phi(r),$$

then

$$\phi(r)^2 \geq 0$$

for all r. A small defensive density is then added so that the proposal density is positive wherever the target might be positive.

11 Squared-TT Density, Defensive Reference, And Normalizer

Source units S3.1-Eq13 and Lemma 1. The question answered here is: how can a TT approximation be forced to define a nonnegative density whose support covers the target?

Zhao–Cui Eq. (13) becomes the following. Let $\pi(r) \geq 0$ be an unnormalized target on a domain $\Omega \subseteq \mathbb{R}^D$, with normalizer

$$Z = \int_\Omega \pi(r)\,\mathrm{d}r.$$

Let $\lambda(r)$ be a normalized reference density on Ω,

$$\lambda(r) \geq 0, \qquad \int_\Omega \lambda(r)\,\mathrm{d}r = 1,$$

and choose $\tau > 0$. Fit a TT $\phi(r)$ to $\sqrt{\pi(r)}$. Define

$$\widehat{\pi}(r) = \phi(r)^2 + \tau\lambda(r), \qquad \widehat{Z} = \int_\Omega \widehat{\pi}(r)\,\mathrm{d}r, \qquad \widehat{p}(r) = \frac{\widehat{\pi}(r)}{\widehat{Z}}. \tag{S1}$$

Since λ is normalized,

$$\widehat{Z} = \int_\Omega \phi(r)^2\,\mathrm{d}r + \tau. \tag{S2}$$

The defensive term has two jobs. First, it keeps the approximation positive even if ϕ is zero somewhere. Second, if π/λ is bounded, then

$$\frac{p(r)}{\widehat{p}(r)} = \frac{\pi(r)/Z}{\widehat{\pi}(r)/\widehat{Z}} = \frac{\widehat{Z}}{Z}\frac{\pi(r)}{\phi(r)^2 + \tau\lambda(r)} \leq \frac{\widehat{Z}}{Z\tau}\frac{\pi(r)}{\lambda(r)}. \tag{S3}$$

This is the importance-sampling reason for the defensive density: the true target is absolutely continuous with respect to the approximation when the reference covers the target support.

Zhao–Cui's Lemma 1 ties the square-root fit to density error. In the notation above, suppose

$$\|\phi - \sqrt{\pi}\|_{L^2} \leq \varepsilon, \qquad \sqrt{\tau} \leq \|\phi - \sqrt{\pi}\|_{L^2}. \tag{S4}$$

Then the square-root error of the defensive approximation satisfies

$$\|\sqrt{\widehat{\pi}} - \sqrt{\pi}\|_{L^2} \leq 2\varepsilon, \tag{S5}$$

and the normalized Hellinger distance obeys

$$D_H(\widehat{p}, p) \leq \frac{\sqrt{2}}{\sqrt{Z}}\|\sqrt{\widehat{\pi}} - \sqrt{\pi}\|_{L^2} \leq \frac{2\sqrt{2}\varepsilon}{\sqrt{Z}}. \tag{S6}$$

For this note, the important lesson is local: fitting the square root in L^2 controls the normalized density in Hellinger distance after normalization. The lemma does not say ranks will be small, nor does it make an adaptive implementation globally differentiable.

Local assumption box for Lemma 1. P18 uses only the following local content: if the square-root fit is small in L^2, then the normalized defensive density is close in Hellinger distance under the stated support and finite-normalizer conditions. P18 does not re-prove the external Cui–Dolgov theorem, does not claim a rank bound, and does not claim adaptive differentiability.

Implementation contract for S3.1-Eq13. Inputs are a square-root target evaluator, a reference density λ, a defensive mass τ, and a fitted TT ϕ. Outputs are $\widehat{\pi}$, $\widehat{Z}$, and $\widehat{p}$. Persisted state is ϕ's cores, λ, τ, and $\widehat{Z}$. Diagnostics are positivity, finite $\widehat{Z}$, support coverage by λ, and the defensive ratio $\tau/\widehat{Z}$.

12 Squared-TT Marginalization And Mass Matrices

Source unit S3.1-Proposition2. This proposition answers: how do we marginalize a squared TT without expanding the square into a full tensor?

The square changes the marginalization algebra. Let

$$\phi(r) = H_1(r_1) \cdots H_D(r_D), \qquad H_k(r_k) \in \mathbb{R}^{R_{k-1} \times R_k}.$$

For a split after coordinate k, define

$$H_{\le k}(r_{\le k}) = H_1(r_1) \cdots H_k(r_k) \in \mathbb{R}^{1 \times R_k},$$

and

$$H_{>k}(r_{>k}) = H_{k+1}(r_{k+1}) \cdots H_D(r_D) \in \mathbb{R}^{R_k \times 1}.$$

Then

$$\phi(r) = H_{\le k}(r_{\le k}) H_{>k}(r_{>k}).$$

The marginal square term is

$$\begin{aligned} \int \phi(r)^2 \, \mathrm{d}r_{>k} &= \int \left[H_{\le k}(r_{\le k}) H_{>k}(r_{>k}) \right]^2 \, \mathrm{d}r_{>k} \\ &= \sum_{a=1}^{R_k} \sum_{b=1}^{R_k} H_{\le k,a}(r_{\le k}) \left[\int H_{>k,a}(r_{>k}) H_{>k,b}(r_{>k}) \, \mathrm{d}r_{>k} \right] H_{\le k,b}(r_{\le k}). \end{aligned} \tag{M1}$$

Define the right mass matrix

$$M_{>k}[a,b] = \int H_{>k,a}(r_{>k}) H_{>k,b}(r_{>k}) \, \mathrm{d}r_{>k}. \tag{M2}$$

This matrix is positive semidefinite. If $M_{>k} = L_{>k} L_{>k}^\top$ is a Cholesky or square-root factorization, then

$$\int \phi(r)^2 \, \mathrm{d}r_{>k} = \sum_{\gamma=1}^{R_k} \left[\sum_{a=1}^{R_k} H_{\le k,a}(r_{\le k}) L_{>k}[a,\gamma] \right]^2. \tag{M3}$$

Including the defensive reference, if $\lambda(r) = \prod_{j=1}^D \lambda_j(r_j)$, gives

$$\widehat{p}(r_{\le k}) = \frac{\sum_{\gamma=1}^{R_k} \left[\sum_{a=1}^{R_k} H_{\le k,a}(r_{\le k}) L_{>k}[a,\gamma] \right]^2 + \tau \prod_{j=1}^k \lambda_j(r_j)}{\widehat{Z}}. \tag{M4}$$

This is Zhao–Cui Eq. (14) in BayesFilter notation.

The practical recursion starts from the right end. Set $M_{>D} = 1$. Moving from $j = D$ down to 1, compute

$$M_{>j-1} = \int H_j(r_j) M_{>j} H_j(r_j)^\top \, \mathrm{d}r_j. \tag{M5}$$

In basis coefficients this integral is finite-dimensional. If

$$H_j^{a,b}(r_j) = \sum_\ell C_j[a,\ell,b] \psi_{j,\ell}(r_j),$$

and

$$B_j[\ell, \ell'] = \int \psi_{j,\ell}(r_j)\psi_{j,\ell'}(r_j)\,\mathrm{d}r_j,$$

then

$$M_{>j-1}[a, a'] = \sum_{b,b'=1}^{R_j} \sum_{\ell,\ell'} C_j[a, \ell, b]\, M_{>j}[b, b']\, C_j[a', \ell', b']\, B_j[\ell, \ell']. \tag{M6}$$

This formula is the bridge from the paper to code: marginalization stores mass matrices or their square roots and never enumerates a full tensor grid.

Implementation contract for S3.1-Proposition2. Inputs are TT cores, basis mass matrices B_j, defensive reference factors λ_j, and τ. Outputs are marginal density evaluators and mass contractions $M_{>k}$ or their Cholesky factors. Persisted state is the list of mass matrices needed by later conditional CDFs. Diagnostics are symmetry error $\|M - M^\top\|$, Cholesky failure, negative eigenvalues beyond tolerance, and normalizer mismatch between independent contraction paths.

Local assumption box for Proposition 2. P18 uses the finite-dimensional contraction identity for a squared TT with integrable basis products and positive semidefinite mass matrices. The note derives the contraction algebra explicitly in (M1)–(M9). It does not claim that every numerical Cholesky factorization is stable; that is a diagnostic obligation.

12.1 A Small Three-Coordinate Marginalization

Take $D = 3$, and suppose

$$\phi(r_1, r_2, r_3) = H_1(r_1)H_2(r_2)H_3(r_3).$$

If $H_1 \in \mathbb{R}^{1\times R_1}$, $H_2 \in \mathbb{R}^{R_1\times R_2}$, and $H_3 \in \mathbb{R}^{R_2\times 1}$, then marginalizing out r_3 gives

$$\int \phi(r_1, r_2, r_3)^2\,\mathrm{d}r_3 = H_1(r_1)H_2(r_2)\left[\int H_3(r_3)H_3(r_3)^\top\,\mathrm{d}r_3\right] H_2(r_2)^\top H_1(r_1)^\top. \tag{M7}$$

The bracketed quantity is an $R_2 \times R_2$ matrix. It is not a function of r_1 or r_2; it is a stored contraction. If we also marginalize out r_2, then

$$M_{>1} = \int H_2(r_2)\left[\int H_3(r_3)H_3(r_3)^\top\,\mathrm{d}r_3\right] H_2(r_2)^\top\,\mathrm{d}r_2, \tag{M8}$$

and the one-coordinate marginal is

$$\int \phi(r_1, r_2, r_3)^2\,\mathrm{d}r_2\,\mathrm{d}r_3 = H_1(r_1)M_{>1}H_1(r_1)^\top. \tag{M9}$$

This is exactly what the recursive mass formula (M5) does. The notation in the paper is compact because it assumes the reader already sees this matrix chain. The implementation sees it as repeated multiplication and integration of small matrices.

13 Conditional Densities And KR Maps From Marginal Ratios

Source units S3.1-KR-ratios and Remark 3. The question answered here is: how do marginal densities become a transport map that can sample from the approximate density?

For a normalized density $\widehat{p}(r_1, \ldots, r_D)$, the chain rule gives

$$\widehat{p}(r) = \widehat{p}(r_1) \prod_{k=2}^{D} \widehat{p}(r_k \mid r_{<k}), \qquad \widehat{p}(r_k \mid r_{<k}) = \frac{\widehat{p}(r_{\leq k})}{\widehat{p}(r_{<k})}. \tag{K1}$$

The squared-TT marginal formula (M4) supplies both numerator and denominator. Thus each conditional density is computable from TT contractions.

Define conditional distribution functions

$$F_1(r_1) = \int_{-\infty}^{r_1} \widehat{p}(s_1)\, \mathrm{d}s_1, \tag{K2}$$

and, for $k \geq 2$,

$$F_k(r_k \mid r_{<k}) = \int_{-\infty}^{r_k} \widehat{p}(s_k \mid r_{<k})\, \mathrm{d}s_k. \tag{K3}$$

The lower triangular Knothe–Rosenblatt map is

$$F(r) = \big(F_1(r_1), F_2(r_2 \mid r_1), \ldots, F_D(r_D \mid r_{<D})\big). \tag{K4}$$

If $R \sim \widehat{p}$, then $F(R)$ is uniform on $[0,1]^D$. Conversely, if $\Xi \sim \mathrm{Unif}([0,1]^D)$, then $F^{-1}(\Xi) \sim \widehat{p}$.

The proof is the triangular Jacobian calculation. The Jacobian of F is lower triangular, and

$$\frac{\partial F_k(r_k \mid r_{<k})}{\partial r_k} = \widehat{p}(r_k \mid r_{<k}).$$

Therefore

$$|\det \nabla F(r)| = \prod_{k=1}^{D} \widehat{p}(r_k \mid r_{<k}) = \widehat{p}(r). \tag{K5}$$

The pullback of the uniform density by F is exactly $|\det \nabla F(r)| = \widehat{p}(r)$.

Source unit S3.1-Remark3. Remark 3 answers: what changes if the conditional map is built from the right end rather than from the left end?

The paper also states the reverse-order construction. If instead we compute right-to-left marginals

$$\widehat{p}(r_{\geq k}) = \int \widehat{p}(r)\, \mathrm{d}r_{<k}, \tag{K6}$$

then the conditional densities are

$$\widehat{p}(r_k \mid r_{>k}) = \frac{\widehat{p}(r_{\geq k})}{\widehat{p}(r_{>k})}, \tag{K7}$$

and the upper triangular KR map is

$$F^u(r) = \big(F_1^u(r_1 \mid r_{>1}), \ldots, F_{D-1}^u(r_{D-1} \mid r_D), F_D^u(r_D)\big). \tag{K8}$$

This is not a different density approximation. It is the same density integrated in the opposite direction. Zhao–Cui use the lower map for backward smoothing and the upper map for forward particle proposals.

Implementation contract for Remark 3. Inputs are the same squared-TT density and the choice of reverse integration direction. Outputs are right-to-left mass contractions and upper-triangular conditional CDFs. Persisted state must record the direction, because using lower-map contractions

inside an upper-map proposal silently changes the proposal density. The failure check is endpoint normalization of every reverse conditional CDF.

The computational costs in the paper separate preprocessing from per-sample map evaluation. Building all conditional densities from the squared-TT mass recursions costs

$$O(DpR^3). \tag{K9}$$

For N samples, evaluating conditional densities along a triangular map costs approximately

$$O(NDpR^2), \tag{K10}$$

and constructing/evaluating one-dimensional distribution functions adds basis and root-finding work. If c_{root} is the fixed number of root-finding iterations, inverse-map evaluation scales as

$$O(DpR^3 + NDpR^2 + NDp(\log p + R) + NDp\, c_{\text{root}}). \tag{K11}$$

The formula is less important than the decomposition: build the conditional objects once, then pay a per-sample triangular evaluation cost.

Implementation contract for S3.1-KR-ratios. Inputs are marginal density evaluators from Proposition 2. Outputs are conditional CDF evaluators and inverse-CDF samplers. Persisted state includes integration direction, mass matrices, one-dimensional CDF coefficients, and root-finding tolerances. Diagnostics are monotonicity of each CDF, endpoint values near 0 and 1, and inverse residuals.

Local assumption box for KR exactness. P18 uses the triangular CDF identity under positive conditional densities, well-defined marginal ratios, and invertible one-dimensional conditional CDFs. If a conditional CDF is flat, discontinuous, numerically non-monotone, or unsupported by the reference domain, the KR sampler is not valid until that failure is fixed.

14 Zhao–Cui Algorithm 2, Fully Annotated

Source units S3.2-Algorithm2(a)–(c), Eq15–Eq16. Algorithm 2 answers: how does the sequential recursion change when the fitted object is the square root and the stored density is squared?

Algorithm 2 is Algorithm 1 with a square-root TT and a defensive density. At time $t-1$, assume the previous filter approximation is evaluable as

$$\widehat{p}_{t-1}(x_{t-1}, \alpha).$$

The pointwise unnormalized target, Zhao–Cui Eq. (15), is

$$q_t(x_t, \alpha, x_{t-1}) = \widehat{p}_{t-1}(x_{t-1}, \alpha) f_\alpha(x_t \mid x_{t-1}) g_\alpha(y_t \mid x_t). \tag{A2-1}$$

Fit a TT to the square root:

$$\sqrt{q_t(x_t, \alpha, x_{t-1})} \approx \phi_t(x_t, \alpha, x_{t-1}). \tag{A2-2}$$

Then define Zhao–Cui Eq. (16):

$$\widehat{q}_t(x_t, \alpha, x_{t-1}) = \phi_t(x_t, \alpha, x_{t-1})^2 + \tau_t \lambda_t(x_t) \lambda_\alpha(\alpha) \lambda_{t-1}(x_{t-1}). \tag{A2-3}$$

The approximate normalizer is

$$\widehat{Z}_t = \int \widehat{q}_t(x_t, \alpha, x_{t-1})\,\mathrm{d}x_t\ \mathrm{d}\alpha\ \mathrm{d}x_{t-1}. \tag{A2-4}$$

The next normalized approximate filter is

$$\widehat{p}_t(x_t, \alpha) = \frac{\int \widehat{q}_t(x_t, \alpha, x_{t-1})\,\mathrm{d}x_{t-1}}{\widehat{Z}_t}. \tag{A2-5}$$

Everything is now nonnegative. The approximation risk has moved: the method must fit the square root accurately enough, choose a defensive term that is not too small for stability or too large for accuracy, and keep TT ranks manageable.

Mini implementation contract. Inputs: previous retained density evaluator, model evaluators, reference factors $\lambda_t, \lambda_\alpha, \lambda_{t-1}$, τ_t, basis/rank/fitting controls, and observation y_t. Outputs: square-root TT ϕ_t, nonnegative density evaluator $\widehat{q}_t$, normalizer $\widehat{Z}_t$, retained filter $\widehat{p}_t(x_t, \alpha)$, and mass contractions for conditional maps. Persisted state: all of those objects. Diagnostics: square-root fit residual, defensive mass ratio, $\widehat{Z}_t > 0$, rank growth, and Cholesky/mass-matrix health.

15 Forward Conditional Map And Particle-Filter Correction

Source units S3.3-Eq20–Eq23 and Algorithm3. These source units answer: how does the squared-TT approximation become a particle proposal, and how is approximation bias corrected?

The same $\widehat{q}_t(x_t, \alpha, x_{t-1})$ defines a conditional density for the new state given the old state and parameter:

$$\widehat{p}_t(x_t \mid \alpha, x_{t-1}, y_{1:t}) = \frac{\widehat{q}_t(x_t, \alpha, x_{t-1})}{\int \widehat{q}_t(s, \alpha, x_{t-1})\,\mathrm{d}s}. \tag{F1}$$

Constructing the corresponding upper triangular conditional KR map gives a proposal sampler:

$$\widehat{X}_t^{(i)} = (F_{t,t}^u)^{-1}\left(\Xi_t^{(i)} \mid \widehat{\alpha}^{(i)}, \widehat{x}_{t-1}^{(i)}\right), \qquad \Xi_t^{(i)} \sim \mathrm{Unif}([0,1]^{m_x}). \tag{F2}$$

This is Zhao–Cui Eq. (21) in notation adapted to BayesFilter.

The proposal density for the full path after sampling is

$$\widehat{p}_t(x_t \mid \alpha, x_{t-1}, y_{1:t}) p(\alpha, x_{0:t-1} \mid y_{1:t-1}), \tag{F3}$$

which corresponds to Zhao–Cui Eq. (22). The exact posterior recursion is

$$p(\alpha, x_{0:t} \mid y_{1:t}) \propto p(\alpha, x_{0:t-1} \mid y_{1:t-1}) f_\alpha(x_t \mid x_{t-1}) g_\alpha(y_t \mid x_t).$$

Dividing exact target by proposal gives the unnormalized correction factor

$$\omega_f(\alpha, x_{t-1:t}) = \frac{f_\alpha(x_t \mid x_{t-1}) g_\alpha(y_t \mid x_t)}{\widehat{p}_t(x_t \mid \alpha, x_{t-1}, y_{1:t})}. \tag{F4}$$

This is Zhao–Cui Eq. (23). The correction is conceptually important: the TT proposal can be biased, but if its conditional density is evaluable and covers the target, importance weights can correct expectations.

With incoming normalized weights $W_{t-1}^{(i)}$, the particle-filter weight update is

$$\widetilde{W}_t^{(i)} = W_{t-1}^{(i)} \omega_f(\widehat{\alpha}^{(i)}, \widehat{x}_{t-1:t}^{(i)}), \tag{F5}$$

followed by

$$W_t^{(i)} = \frac{\widetilde{W}_t^{(i)}}{\sum_{j=1}^N \widetilde{W}_t^{(j)}}. \tag{F6}$$

Algorithm 3 is therefore an ordinary importance sampler whose proposal is defined by the upper conditional KR map of the TT approximation. The implementation stores the conditional-map object, the sampled $x_t^{(i)}$, the proposal density evaluator, and the normalized weights. A standard failure diagnostic is effective sample size,

$$\mathrm{ESS}_t = \frac{1}{\sum_{i=1}^N (W_t^{(i)})^2}. \tag{F7}$$

Low ESS means that the TT proposal is not close enough to the exact recursive posterior for stable correction.

Mini implementation contract. Inputs: weighted particles at time $t-1$, the upper conditional map, model density evaluators, and proposal density evaluator. Outputs: new particles and normalized weights. Persisted state: particles, weights, ESS, and optional resampling history. Operation: inverse triangular sampling plus importance-weight update. Diagnostics: ESS, infinite/zero proposal density, root-finding failure, and weight underflow.

16 Backward Conditional Map, Path Estimation, And Smoothing

Source units S3.4-Eq24–Eq26, Figure 1, and Algorithm4. These source units answer: how can the same sequence of local TT approximations produce full paths and smoothed old states after all data are observed?

The lower conditional map goes the other direction:

$$\widehat{p}_t(x_{t-1} \mid x_t, \alpha, y_{1:t}) = \frac{\widehat{q}_t(x_t, \alpha, x_{t-1})}{\int \widehat{q}_t(x_t, \alpha, s)\,\mathrm{d}s}. \tag{B1}$$

The corresponding inverse conditional KR map samples

$$\widehat{X}_{t-1}^{(i)} = (F_{t,t-1}^l)^{-1}\left(\Xi_{t-1}^{(i)} \mid \widehat{X}_t^{(i)}, \widehat{\alpha}^{(i)}\right). \tag{B2}$$

This is Zhao–Cui Eq. (24).

Starting at final time T, draw

$$(\widehat{X}_T, \widehat{\alpha}, \widehat{X}_{T-1}) \sim \widehat{p}_T(x_T, \alpha, x_{T-1} \mid y_{1:T}),$$

then move backward using (B2). The resulting approximate path density factors as Zhao–Cui Eq. (25):

$$\widehat{p}(\alpha, x_{0:T} \mid y_{1:T}) = \widehat{p}_T(x_T, \alpha, x_{T-1} \mid y_{1:T}) \prod_{t=1}^{T-1} \widehat{p}_t(x_{t-1} \mid x_t, \alpha, y_{1:t}). \tag{B3}$$

The exact smoothing posterior has the same Markov factorization form with exact conditionals:

$$p(\alpha, x_{0:T} \mid y_{1:T}) = p(x_T, \alpha, x_{T-1} \mid y_{1:T}) \prod_{t=1}^{T-1} p(x_{t-1} \mid x_t, \alpha, y_{1:t}). \tag{B4}$$

The approximation can therefore be corrected by an importance ratio. The unnormalized backward weight, Zhao–Cui Eq. (26), is

$$\omega_b(\alpha, x_{0:T}) = \frac{p(\alpha)p_\alpha(x_0)\prod_{t=1}^{T} f_\alpha(x_t \mid x_{t-1})g_\alpha(y_t \mid x_t)}{\widehat{p}_T(x_T, \alpha, x_{T-1} \mid y_{1:T})\prod_{t=1}^{T-1}\widehat{p}_t(x_{t-1} \mid x_t, \alpha, y_{1:t})}. \tag{B5}$$

The numerator is the exact unnormalized path posterior. The denominator is the proposal density generated by the backward TT maps.

Algorithm 4 normalizes the backward weights in the same way:

$$W^{(i)} = \frac{\omega_b(\widehat{\alpha}^{(i)}, \widehat{x}_{0:T}^{(i)})}{\sum_{j=1}^{N}\omega_b(\widehat{\alpha}^{(j)}, \widehat{x}_{0:T}^{(j)})}. \tag{B6}$$

The paper emphasizes a Markov identity used by the backward recursion:

$$p(x_{t-1} \mid \alpha, x_t, y_{1:t}) = p(x_{t-1} \mid \alpha, x_t, y_{1:T}), \qquad t < T. \tag{B7}$$

The reason is conditional independence. Once (α, x_t) are known, the future observations $y_{t+1:T}$ depend on the past state x_{t-1} only through x_t. This is why a conditional object built at time t can be used inside a final-time smoothing path.

Zhao–Cui also give the computational reason for the variable order (x_t, α, x_{t-1}). Marginalizing a squared TT from an end costs

$$O(pR^3) \quad \text{per coordinate,} \tag{B8}$$

because it updates one boundary mass matrix. Marginalizing a coordinate in the middle can cost

$$O(pR^6) \quad \text{per coordinate,} \tag{B9}$$

because left and right rank couplings must be paired. The order (x_t, α, x_{t-1}) lets Algorithm 2 integrate x_{t-1} from the right end and lets the smoothing map reuse the same direction. The upper map for particle filtering goes in the opposite direction and therefore incurs an additional $O(mpR^3)$-type preprocessing cost.

Mini implementation contract. Inputs: final-time approximation, lower conditional maps for all times, model density evaluators, and reference uniform samples. Outputs: path samples and normalized smoothing weights. Persisted state: map sequence, samples, weights, and ESS. Operation: final joint sample, then backward inverse-map sampling. Diagnostics: backward proposal support, weight variance, root residuals, and mismatch between source-time conditionals and final-time path use.

17 Error Propagation: What It Proves And What It Does Not Prove

Section 4 of Zhao–Cui explains how local approximation errors propagate. The key identity is the triangle inequality. Let π_t be the exact unnormalized joint density of (x_t, α, x_{t-1}) at time t, q_t the propagated density using the previous approximation, and $\widehat{\pi}_t$ the new TT approximation. Then

$$D(\widehat{\pi}_t, \pi_t) \le D(\widehat{\pi}_t, q_t) + D(q_t, \pi_t). \tag{E1}$$

The first term is the new fit error. The second is inherited from the previous time step. Under bounded likelihood or bounded prediction-likelihood conditions, the inherited error is multiplied by a finite constant.

For squared TT, Zhao–Cui use an L^2 error on square roots because it controls Hellinger distance after normalization. The practical message is precise but modest: if every local square-root approximation is accurate enough and the model does not amplify errors too severely, the sequence of approximate densities can remain controlled. This is not a theorem that ranks will stay small, that the companion implementation is globally differentiable, or that a particular high-dimensional model will be accurate without diagnostics.

18 Preconditioning

Source units S5.1-Eq30–Eq33. These source units answer: how can a hard target density be transformed into a less concentrated residual before fitting a TT?

The direct target $q_t(x_t, \alpha, x_{t-1})$ can be sharply concentrated. A TT may then need high rank. Zhao–Cui Section 5 introduces a change of variables that makes the function closer to a product reference density before fitting.

Let

$$r = (x_t, \alpha, x_{t-1}), \qquad u = T_t(r),$$

and let $\eta(u) = \eta_1(u_1) \cdots \eta_D(u_D)$ be a product reference density. Choose a bridging density $\rho_t(r)$ that is easier to approximate than $q_t(r)$, and construct T_t so that the pushforward of ρ_t is close to η:

$$(T_t)_{\#}\rho_t(u) = \rho_t(T_t^{-1}(u)) \left|\det \nabla T_t^{-1}(u)\right| \propto \eta(u). \tag{P1}$$

This is Zhao–Cui Eq. (30).

The pushforward of q_t is

$$(T_t)_{\#}q_t(u) = q_t(T_t^{-1}(u)) \left|\det \nabla T_t^{-1}(u)\right|.$$

Using (P1), the Jacobian factor is proportional to $\eta(u)/\rho_t(T_t^{-1}(u))$, so the function to fit is

$$q_t^\sharp(u) = \frac{q_t(T_t^{-1}(u))}{\rho_t(T_t^{-1}(u))}\eta(u). \tag{P2}$$

This is Zhao–Cui Eq. (31). If ρ_t captures the main shape of q_t, then q_t/ρ_t is flatter than q_t, and TT ranks may be smaller.

In density language, the preconditioner should move the hard geometry into the map T_t. The residual $q_t^\sharp$ then asks the TT to learn only what the bridge missed. The implementation failure mode is division by a bridge that is too small: $\rho_t(T_t^{-1}(u))$ or $\widehat{\rho}_t(T_t^{-1}(u))$ must not vanish on target-relevant points.

Fit a square-root TT in u-space:

$$\sqrt{q_t^\sharp(u)} \approx \phi_t^\sharp(u),$$

and define

$$\widehat{\nu}_t^\sharp(u) = \phi_t^\sharp(u)^2 + \tau_t^\sharp \eta(u). \tag{P3}$$

This is Zhao–Cui Eq. (32). Pulling the approximation back to physical coordinates gives

$$\widehat{\pi}_t(r) = \frac{\widehat{\nu}_t^\sharp(T_t(r))}{\eta(T_t(r))}\rho_t(r). \tag{P4}$$

This is Zhao–Cui Eq. (33). The normalizer is unchanged by the change of variables:

$$\int \widehat{\pi}_t(r)\,\mathrm{d}r = \int \widehat{\nu}_t^{\sharp}(u)\,\mathrm{d}u. \tag{P5}$$

Implementation contract for S5.1-Eq30–Eq33. Inputs are q_t, bridge ρ_t or $\widehat{\rho}_t$, map T_t, reference density η, and residual TT $\widehat{\nu}_t^{\sharp}$. Outputs are a physical-coordinate approximate density evaluator $\widehat{\pi}_t$, a normalizer, and later marginal/conditional map objects. Persisted state: map evaluators, inverse map evaluators, log-Jacobians or density ratios, bridge TT, residual TT, and normalizer. Diagnostics: bridge support ratio, map invertibility, normalizer equality between physical and reference coordinates, and residual rank.

Source unit S5.2-Gaussian-bridge. This source unit answers: what is the simplest concrete preconditioner?

A simple bridge is Gaussian. Estimate a mean and covariance for r, set $\rho_t = \mathcal{N}(\mu_t, \Sigma_t)$, and use an affine Gaussian whitening map. If

$$\Sigma_t = L_t L_t^{\top} \tag{P6a}$$

is a Cholesky factorization with L_t lower triangular, then the lower linear KR whitening map is

$$T_t^{\ell}(r) = L_t^{-1}(r - \mu_t), \tag{P6b}$$

and $T_t^{\ell}(R) \sim N(0, I)$ when $R \sim N(\mu_t, \Sigma_t)$. The reference density η is then the standard Gaussian product density. The implementation stores μ_t, L_t, triangular solves for L_t^{-1}, and the log determinant

$$\log|\det \nabla T_t^{\ell}| = -\sum_j \log L_t[j,j]. \tag{P6c}$$

An upper triangular linear map is obtained by permuting or reversing the coordinate order before applying the same whitening idea.

Implementation contract for S5.2. Inputs: particles or quadrature samples used to estimate μ_t, Σ_t. Outputs: triangular factor L_t, whitening map T_t^{ℓ}, inverse map $r = \mu_t + L_t u$, and log determinant. Persisted state: μ_t, L_t and any covariance regularization. Diagnostics: positive-definiteness of Σ_t, conditioning of L_t, and whether the Gaussian bridge assigns negligible density to target-relevant samples.

Source unit S5.3-Eq34–Eq35. This source unit answers: how can the bridge retain nonlinear structure from the previous filter, transition, and likelihood?

A nonlinear bridge uses powers of the three factors in q_t:

$$\rho_t(r) = \widehat{p}_{t-1}(x_{t-1}, \alpha)^{\beta_\pi} f_\alpha(x_t \mid x_{t-1})^{\beta_f} g_\alpha(y_t \mid x_t)^{\beta_g}, \qquad \beta_\pi, \beta_f, \beta_g \in [0,1]. \tag{P6}$$

This is Zhao–Cui Eq. (34). A squared-TT approximation of this bridge is

$$\widehat{\rho}_t(r) = \phi_{\rho,t}(r)^2 + \tau_{\rho,t}\lambda(r). \tag{P7}$$

This is Zhao–Cui Eq. (35). The same algebra as (P4) applies with ρ_t replaced by $\widehat{\rho}_t$.

Implementation contract for S5.3. Inputs: exponents $\beta_\pi, \beta_f, \beta_g$, previous filter evaluator, transition evaluator, likelihood evaluator, and bridge TT controls. Outputs: $\widehat{\rho}_t$ and its KR map. Persisted state: exponents, bridge TT cores, defensive mass, and bridge mass contractions. Diagnostics: bridge fit residual, bridge rank, support coverage, and whether the powered factors are finite at fitting points.

18.1 The Five Densities In The Preconditioned Step

Source unit S5.3-density-chain. This source unit expands the compact Section 5 notation into the five densities an implementer must distinguish.

The preconditioned step is easy to lose because the paper moves between physical coordinates $r = (x_t, \theta, x_{t-1})$, reference coordinates $u = (u_t, u_\theta, u_{t-1})$, an exact bridge, an approximate bridge, and a residual density. The complete chain is as follows.

First, there is the exact target we ultimately want to approximate:

$$q_t(r) = \widehat{\pi}_{t-1}(x_{t-1}, \theta) f_\theta(x_t \mid x_{t-1}) g_\theta(y_t \mid x_t). \tag{P7a}$$

Second, there is an exact bridge $\rho_t(r)$. It is not the final answer. It is a simpler density chosen to resemble the main shape of q_t. In the tempered construction it is the powered product in (P6). In the Gaussian construction it is $N(\mu_t, \Sigma_t)$.

Third, because the exact bridge may still be nonseparable, Algorithm 5(b.1) approximates the bridge itself by a squared TT:

$$\sqrt{\rho_t(r)} \approx \phi_{\rho,t}(r), \qquad \widehat{\rho}_t(r) = \phi_{\rho,t}(r)^2 + \tau_{\rho,t}\lambda(r). \tag{P7b}$$

This is the first line of Algorithm 5 in mathematical form. The implementation stores the bridge TT cores, the defensive mass $\tau_{\rho,t}$, the reference density λ, the bridge normalizer, and the mass contractions needed to construct conditional maps from $\widehat{\rho}_t$.

Fourth, build the preconditioning map from the bridge approximation. Let

$$T_t : \ r \mapsto u \tag{P7c}$$

be a KR-based map satisfying

$$(T_t)_\# \widehat{\rho}_t(u) \propto \eta(u). \tag{P7d}$$

Using $\widehat{\rho}_t$, not the unavailable exact bridge, the residual target to approximate in reference coordinates is

$$q_t^\sharp(u) = \frac{q_t(T_t^{-1}(u))}{\widehat{\rho}_t(T_t^{-1}(u))}\eta(u). \tag{P7e}$$

This formula answers the question "what remains after the bridge has removed the main shape?" If the bridge is good, the ratio $q_t/\widehat{\rho}_t$ is flatter than q_t, so the residual may need smaller TT ranks.

Fifth, approximate the square root of the residual:

$$\sqrt{q_t^\sharp(u)} \approx \phi_t^\sharp(u), \qquad \widehat{\nu}_t^\sharp(u) = \phi_t^\sharp(u)^2 + \tau_t^\sharp \eta(u). \tag{P7f}$$

Finally pull this residual approximation back to physical coordinates:

$$\widehat{\pi}_t(r) = \frac{\widehat{\nu}_t^\sharp(T_t(r))}{\eta(T_t(r))}\widehat{\rho}_t(r). \tag{P7g}$$

To check the algebra, substitute $u = T_t(r)$. Since $(T_t)_\#\widehat{\rho}_t \propto \eta$, the factor $\widehat{\rho}_t(r)/\eta(T_t(r))$ restores the physical-coordinate density scale. The normalizer is computed in reference space:

$$\int \widehat{\pi}_t(r)\,\mathrm{d}r = \int \widehat{\nu}_t^\sharp(u)\,\mathrm{d}u. \tag{P7h}$$

Algorithm 5 stores exactly the objects in this chain:

$$\widehat{\rho}_t, \quad T_t, \quad q_t^\sharp \text{ evaluator}, \quad \widehat{\nu}_t^\sharp, \quad \widehat{\pi}_t \text{ evaluator}, \quad \text{marginal and conditional maps.} \tag{P7i}$$

This is the preconditioning story in one line of thought: approximate the bridge, use the bridge to flatten the target, approximate the flattened residual, then pull the residual back.

Failure checks for the five-density chain. Verify all of the following:

$$q_t(r) \geq 0, \qquad \widehat{\rho}_t(r) > 0 \text{ on fitted target points}, \qquad q_t^\sharp(u) \geq 0, \tag{P7j}$$

$$0 < \int \widehat{\nu}_t^\sharp(u)\,\mathrm{d}u < \infty, \qquad \left| \int \widehat{\pi}_t(r)\,\mathrm{d}r - \int \widehat{\nu}_t^\sharp(u)\,\mathrm{d}u \right| \leq \varepsilon_{\mathrm{norm}}. \tag{P7k}$$

These are not cosmetic checks. They assess whether the density chain still defines the same scalar after moving between coordinate systems.

Source unit S5.4-Algorithm5(b.1). Algorithm 5(b.1) answers: what approximation is built before the residual target is fitted?

The bridge approximation is $\widehat{\rho}_t$, already defined in (P7b). Its mathematical input-output contract is

$$(r \mapsto \rho_t(r), \lambda, \tau_{\rho,t}, \text{basis}, \text{ranks}) \mapsto (\widehat{\rho}_t, \text{bridge mass contractions}). \tag{P7l}$$

Failure checks: bridge fit residual, positivity of $\widehat{\rho}_t$, and nonzero bridge values on target fitting points.

Source unit S5.4-Algorithm5(b.2). Algorithm 5(b.2) answers: how does the bridge approximation become a lower preconditioning map and a bridge marginal?

Build a KR map R_t from $\widehat{\rho}_t$ to a uniform variable:

$$(R_t)_\# \widehat{\rho}_t(\xi_t, \xi_\theta, \xi_{t-1}) \propto \mathrm{Unif}(\xi_t, \xi_\theta, \xi_{t-1}). \tag{P8}$$

If D is a diagonal coordinate map satisfying

$$D_\# \eta(\xi_t, \xi_\theta, \xi_{t-1}) = \mathrm{Unif}(\xi_t, \xi_\theta, \xi_{t-1}), \tag{P9}$$

then

$$T_t = D^{-1} \circ R_t \tag{P10}$$

pushes the bridge approximation to the product reference η:

$$(T_t)_\# \widehat{\rho}_t(u_t, u_\theta, u_{t-1}) \propto \eta(u_t, u_\theta, u_{t-1}). \tag{P11}$$

For the lower-triangular version, write

$$T_t^\ell(x_t, \theta, x_{t-1}) = \big(T_{t,t}^\ell(x_t), T_{t,\theta}^\ell(\theta \mid x_t), T_{t,t-1}^\ell(x_{t-1} \mid x_t, \theta)\big). \tag{P11a}$$

The same bridge mass contractions also provide the bridge marginal

$$\widehat{\rho}_t(x_t, \theta) = \int \widehat{\rho}_t(x_t, \theta, x_{t-1})\,\mathrm{d}x_{t-1}. \tag{P11b}$$

Mathematical input-output contract:

$$\widehat{\rho}_t \mapsto (T_t^\ell, (T_t^\ell)^{-1}, \widehat{\rho}_t(x_t, \theta), T_{t,t-1}^\ell). \tag{P11c}$$

Failure checks: monotone conditional CDFs, inverse-map residuals, and agreement between the bridge marginal and direct bridge contraction.

Source unit S5.4-Algorithm5(b.3). Algorithm 5(b.3) answers: after the bridge map is available, what residual density is fitted in reference coordinates?

With this nonlinear preconditioner, the residual target becomes

$$q_t^\sharp(u) = \frac{q_t(T_t^{-1}(u))}{\widehat{\rho}_t(T_t^{-1}(u))}\eta(u), \tag{P12}$$

and the pulled-back unnormalized posterior approximation is

$$\widehat{\pi}_t(r) = \frac{\widehat{\nu}_t^\sharp(T_t(r))}{\eta(T_t(r))}\widehat{\rho}_t(r). \tag{P13}$$

Mathematical input-output contract:

$$(q_t, T_t^{-1}, \widehat{\rho}_t, \eta, \text{basis}, \text{ranks}) \mapsto (\widehat{\nu}_t^\sharp, S_t^\ell, \text{residual mass contractions}). \tag{P13a}$$

Failure checks: finite ratio $q_t/\widehat{\rho}_t$, residual rank growth, and normalizer $\int \widehat{\nu}_t^\sharp(u)\,\mathrm{d}u \in (0,\infty)$.

Source unit S5.4-Algorithm5(c.1). Algorithm 5(c.1) answers: what residual marginal is needed before returning to physical coordinates?

Use the lower triangular decompositions

$$S_t^\ell(u_t, u_\theta, u_{t-1}) = \begin{bmatrix} S_{t,t}^\ell(u_t) \\ S_{t,\theta}^\ell(u_\theta \mid u_t) \\ S_{t,t-1}^\ell(u_{t-1} \mid u_t, u_\theta) \end{bmatrix}, \qquad T_t^\ell(x_t, \theta, x_{t-1}) = \begin{bmatrix} T_{t,t}^\ell(x_t) \\ T_{t,\theta}^\ell(\theta \mid x_t) \\ T_{t,t-1}^\ell(x_{t-1} \mid x_t, \theta) \end{bmatrix}. \tag{P14}$$

Step (c.1) integrates the last block (u_{t-1}) of $\widehat{\nu}_t^\sharp$ to obtain

$$\widehat{\nu}_t^\sharp(u_t, u_\theta) = \int \widehat{\nu}_t^\sharp(u_t, u_\theta, u_{t-1})\,\mathrm{d}u_{t-1}. \tag{P15}$$

This is an endpoint squared-TT marginalization in u-space, so it uses the same mass-matrix algebra as Proposition 2. It also creates the residual lower conditional map $S_{t,t-1}^\ell(u_{t-1} \mid u_t, u_\theta)$.

Source unit S5.4-Algorithm5(c.2). Algorithm 5(c.2) answers: how does the residual marginal become the retained physical-coordinate filter?

Using

$$(u_t, u_\theta) = \big(T_{t,t}^\ell(x_t), T_{t,\theta}^\ell(\theta \mid x_t)\big), \tag{P16}$$

start from the pulled-back three-block density (P13):

$$\widehat{\pi}_t(x_t, \theta, x_{t-1}) = \frac{\widehat{\nu}_t^\sharp(T_t^\ell(x_t, \theta, x_{t-1}))}{\eta(T_t^\ell(x_t, \theta, x_{t-1}))}\widehat{\rho}_t(x_t, \theta, x_{t-1}). \tag{P16a}$$

Now change variables only in the old-state conditional block. For fixed (x_t, θ), the lower bridge map sends

$$x_{t-1} \mapsto u_{t-1} = T_{t,t-1}^\ell(x_{t-1} \mid x_t, \theta). \tag{P16b}$$

Because T_t^ℓ was built from the bridge approximation, the bridge density decomposes locally. Write

$$u_t = T_{t,t}^\ell(x_t), \qquad u_\theta = T_{t,\theta}^\ell(\theta \mid x_t). \tag{P16b.1}$$

The conditional part of the triangular pushforward identity says that, for fixed (x_t, θ),

$$\widehat{\rho}_t(x_{t-1} \mid x_t, \theta)\,\mathrm{d}x_{t-1} = \eta(u_{t-1} \mid u_t, u_\theta)\,\mathrm{d}u_{t-1}. \tag{P16b.2}$$

Since the reference density factorizes in the transformed coordinates,

$$\eta(u_t, u_\theta, u_{t-1}) = \eta(u_{t-1} \mid u_t, u_\theta)\eta(u_t, u_\theta), \tag{P16b.3}$$

the conditional density in (P16b.2) can be written as

$$\eta(u_{t-1} \mid u_t, u_\theta) = \frac{\eta(T_t^\ell(x_t, \theta, x_{t-1}))}{\eta(T_{t,t}^\ell(x_t), T_{t,\theta}^\ell(\theta \mid x_t))}. \tag{P16b.4}$$

Multiplying (P16b.2) by the bridge marginal $\widehat{\rho}_t(x_t, \theta)$ gives the local bridge decomposition

$$\widehat{\rho}_t(x_t, \theta, x_{t-1})\,\mathrm{d}x_{t-1} = \frac{\widehat{\rho}_t(x_t, \theta)}{\eta(T_{t,t}^\ell(x_t), T_{t,\theta}^\ell(\theta \mid x_t))}\eta(T_t^\ell(x_t, \theta, x_{t-1}))\,\mathrm{d}u_{t-1}, \tag{P16c}$$

where the Jacobian of the conditional map is included in the differential $\mathrm{d}u_{t-1}$. This is the missing middle step in plain words: the three-block bridge density equals its two-block marginal times the conditional bridge density, and the conditional bridge density becomes the conditional reference density after the triangular map. Substituting (P16c) into (P16a) and integrating over x_{t-1} gives

$$\begin{aligned}\widehat{\pi}_t(x_t, \theta \mid y_{1:t}) &= \int \widehat{\pi}_t(x_t, \theta, x_{t-1})\,\mathrm{d}x_{t-1} \\ &= \frac{\widehat{\rho}_t(x_t, \theta)}{\eta(T_{t,t}^\ell(x_t), T_{t,\theta}^\ell(\theta \mid x_t))} \int \widehat{\nu}_t^\sharp(u_t, u_\theta, u_{t-1})\,\mathrm{d}u_{t-1}.\end{aligned} \tag{P16d}$$

Using (P15) with $(u_t, u_\theta) = (T_{t,t}^\ell(x_t), T_{t,\theta}^\ell(\theta \mid x_t))$, the retained marginal density for the next filtering step is

$$\widehat{\pi}_t(x_t, \theta \mid y_{1:t}) = \frac{\widehat{\nu}_t^\sharp\Big(T_{t,t}^\ell(x_t), T_{t,\theta}^\ell(\theta \mid x_t)\Big)}{\eta\Big(T_{t,t}^\ell(x_t), T_{t,\theta}^\ell(\theta \mid x_t)\Big)}\widehat{\rho}_t(x_t, \theta). \tag{P17}$$

This is the source formula in Algorithm 5(c.2) written without relying on paper shorthand. The implementation stores the two maps, their marginal evaluators, and a retained evaluator for $\widehat{\pi}_t(x_t, \theta)$.

As a byproduct, the lower conditional map for path estimation is the last block of the composite map:

$$F_{t,t-1}^\ell(x_{t-1} \mid x_t, \theta) = S_{t,t-1}^\ell\Big(T_{t,t-1}^\ell(x_{t-1} \mid x_t, \theta) \,\Big|\, T_{t,t}^\ell(x_t), T_{t,\theta}^\ell(\theta \mid x_t)\Big). \tag{P18}$$

This identity matters because preconditioning does not eliminate smoothing; it changes which conditional maps must be composed and stored.

Boxed Algorithm 5 dataflow.

Step	Evaluator signature	Persisted objects	Main diagnostic
b.1	$\rho_t(r) \mapsto \widehat{\rho}_t(r)$	bridge TT, $\tau_{\rho,t}, \lambda$	bridge residual/rank
b.2	$\widehat{\rho}_t \mapsto T_t^\ell, (T_t^\ell)^{-1}$	bridge maps, bridge marginal	CDF monotonicity
b.3	$(q_t, T_t^{-1}, \widehat{\rho}_t, \eta) \mapsto \widehat{\nu}_t^\sharp$	residual TT, S_t^ℓ	ratio finite
c.1	$\widehat{\nu}_t^\sharp(u_t, u_\theta, u_{t-1}) \mapsto \widehat{\nu}_t^\sharp(u_t, u_\theta)$	residual marginal, $S_{t,t-1}^\ell$	mass contraction
c.2	$(\widehat{\nu}_t^\sharp, T_t^\ell, \widehat{\rho}_t) \mapsto \widehat{\pi}_t(x_t, \theta)$	retained filter evaluator	normalizer invariance

19 End of Zhao–Cui Annotation and Start of BayesFilter Fixed-Branch Extension

Up to this point the note has annotated Zhao–Cui Sections 1–3 and 5 in source order. The equations above explain the paper's adaptive sequential algorithm: tensor-train approximation, squared-TT nonnegativity, conditional KR maps, particle correction, smoothing, and preconditioning. The next sections are a BayesFilter extension. They freeze approximation choices so that an analytical derivative can differentiate the same scalar computed by the forward pass. Zhao–Cui's adaptive rank, pivot, fitting, preconditioning, and branch choices should not be read as globally differentiable.

Reconciliation: Paper Objects Versus Fixed-Branch Objects

The paper's main sequential-learning density carries a learned static parameter as a random coordinate:

$$r_t^{\mathrm{ZC}} = (x_t, \theta, x_{t-1}). \tag{R1}$$

The fixed-branch likelihood lane conditions on an external parameter value β and differentiates with respect to that external argument:

$$r_t^{\mathrm{FB}} = (x_t, x_{t-1}; \beta). \tag{R2}$$

The semicolon is deliberate: β is not integrated out by the TT density. It is the input at which the approximate likelihood is evaluated.

The exact mapping of objects is:

$$\begin{gathered}
\widehat{\pi}_{t-1}(x_{t-1}, \theta) \mapsto \widehat{p}_{t-1}(x_{t-1}; \beta), \\
f(x_t \mid x_{t-1}, \theta) \mapsto f_\beta(x_t \mid x_{t-1}), \qquad g(y_t \mid x_t, \theta) \mapsto g_\beta(y_t \mid x_t), \\
\phi_t(x_t, \theta, x_{t-1}) \mapsto \phi_t(x_t, x_{t-1}; \beta), \\
\tau_t \lambda(x_t)\lambda(\theta)\lambda(x_{t-1}) \mapsto \tau_t \lambda_t(x_t, x_{t-1}), \\
\widehat{Z}_t = \int \widehat{q}_t(x_t, \theta, x_{t-1}) \mapsto \widehat{Z}_t(\beta) = \int \widehat{q}_t(x_t, x_{t-1}; \beta), \\
\text{adaptive ranks/pivots/maps} \mapsto \text{fixed ranks/points/maps/sweeps.}
\end{gathered} \tag{R3}$$

The first line keeps the retained-filter role but drops the integrated θ coordinate. The second line keeps the model evaluators, evaluated at the external β. The third and fourth lines keep the squared-TT and defensive-density forms in a lower-dimensional state space. The fifth line keeps the evidence-increment role. The last line is the essential change: adaptive choices are replaced by frozen branch choices before differentiation. Thus the fixed-branch section does not differentiate the adaptive Zhao–Cui algorithm. It differentiates a lower-dimensional scalar constructed by applying the same squared-TT filtering algebra after freezing all discrete approximation choices. If one wants a fixed-branch derivative while also learning a static parameter as a random coordinate, one can append that coordinate to the state; the external derivative variable must still be distinguished from the integrated coordinate.

20 Implementable Fixed-Branch Objects And Data Structures

A reader implementing a minimal method must store more than an equation. At time t, a squared-TT filter object contains:

$$\mathcal{B}_t = \{\Omega_{t,k}, \Psi_{t,k}, \psi_{t,k,\ell}, C_{t,k}, R_{t,k}, B_{t,k}, \lambda_{t,k}, \tau_t, c_t, \text{mass contractions}, \text{map data}\}.$$

Here $\Omega_{t,k}$ is the coordinate domain, $\Psi_{t,k}$ maps reference coordinates to physical coordinates, $\psi_{t,k,\ell}$ are basis functions, $C_{t,k}$ are TT cores, $R_{t,k}$ are ranks, $B_{t,k}$ are mass matrices, $\lambda_{t,k}$ are reference densities, τ_t is the defensive mass, and c_t is any log-scaling shift used during fitting.

For the adaptive Zhao–Cui algorithm these objects may change with the data, with samples, and with approximation decisions. For a differentiable same-scalar target, the realized branch must be frozen before differentiation.

20.1 What A Core Object Looks Like

For coordinate k, store

$$C_k \in \mathbb{R}^{R_{k-1}\times p_k\times R_k}.$$

Given a scalar coordinate value r_k, first compute the basis vector

$$b_k(r_k) = \big(\psi_{k,0}(r_k),\dots,\psi_{k,p_k-1}(r_k)\big)^\top.$$

Then form the matrix

$$H_k(r_k)[a,b] = \sum_{\ell=0}^{p_k-1} C_k[a,\ell,b]\, b_k(r_k)[\ell]. \tag{D1}$$

Evaluation of the TT is the loop

$$v_0 = 1, \qquad v_k = v_{k-1}H_k(r_k), \quad k=1,\dots,D, \qquad \widehat{h}(r) = v_D. \tag{D2}$$

The square-density object additionally stores mass contractions

$$M_{>k} = \int H_{k+1}(r_{k+1})\cdots H_D(r_D)H_D(r_D)^\top\cdots H_{k+1}(r_{k+1})^\top \,\mathrm{d}r_{k+1:D}. \tag{D3}$$

These matrices are enough to evaluate marginals from the left. Analogous left-side mass contractions evaluate reverse-order marginals.

20.2 What Must Be Saved For The Next Time Step

After time t, the next step needs to evaluate

$$\widehat{p}_t(x_t;\beta) \quad\text{and}\quad \partial_{\beta_i}\widehat{p}_t(x_t;\beta)$$

at future fitting points. It is not enough to store a cloud of samples or only $\widehat{Z}_t$. A fixed-branch derivative implementation stores:

$$\left\{C_{t,k}, \dot{C}^{(i)}_{t,k}, M_{t,>k}, \dot{M}^{(i)}_{t,>k}, \widehat{Z}_t, \dot{\widehat{Z}}^{(i)}_t, \Psi_t, \lambda_t, \tau_t\right\}. \tag{D4}$$

The derivative of the normalized marginal has the quotient form

$$\partial_{\beta_i}\widehat{p}_t(x_t;\beta) = \frac{\partial_{\beta_i}a_t(x_t;\beta)}{\widehat{Z}_t(\beta)} - \frac{a_t(x_t;\beta)\partial_{\beta_i}\widehat{Z}_t(\beta)}{\widehat{Z}_t(\beta)^2}, \tag{D5}$$

where

$$a_t(x_t;\beta) = \int \widehat{q}_t(x_t,x_{t-1};\beta)\,\mathrm{d}x_{t-1} \tag{D6}$$

is the unnormalized retained numerator. Equation (D5) is the carried-filter derivative needed in (G2) at the next time step.

Lemma 1 (Carried filter object feeds the next target)**.** *Fix the branch at time t. Suppose the stored object contains*

$$\mathcal{F}_t = \{a_t, \dot{a}_t^{(i)}, \widehat{Z}_t, \dot{\widehat{Z}}_t^{(i)}, \Psi_t, \lambda_t, \tau_t\}.$$

Define $\widehat{p}_t$ and $\dot{\widehat{p}}_t^{(i)}$ by (D5). Then the next transformed target and its derivative at any next-step fitting point are computable from $\mathcal{F}_t$, the model density evaluators, and their derivatives.

Proof. At time $t+1$, the fixed-branch target has the form

$$\widetilde{q}_{t+1}(z;\beta) = \widehat{p}_t(x_t(z);\beta) f_\beta(x_{t+1}(z) \mid x_t(z)) g_\beta(y_{t+1} \mid x_{t+1}(z)) J_{t+1}(z).$$

The stored object evaluates $\widehat{p}_t$. The model supplies f_β and g_β, and the branch supplies J_{t+1}. Differentiating gives the product-rule expression (G2) with t replaced by $t+1$. The first term uses $\dot{\widehat{p}}_t^{(i)}$, which is the quotient derivative in (D5). No old samples or external source-paper objects are needed. □

21 One Concrete Branch, Fully Instantiated

This section gives a single implementation lane. It is intentionally narrower than Zhao–Cui's adaptive code. Its purpose is to remove ambiguity. A coder can implement this lane first, then replace pieces with adaptive or higher performance choices later.

21.1 Coordinate Order And Domain

For the fixed-parameter likelihood case, use the two-block coordinate order

$$r_t = (x_t, x_{t-1}) \in \mathbb{R}^2$$

for a scalar state. For vector states, concatenate coordinates as

$$r_t = (x_{t,1}, \ldots, x_{t,m_x}, x_{t-1,1}, \ldots, x_{t-1,m_x}).$$

Choose a fixed interval for each coordinate,

$$r_{t,k} \in [\ell_{t,k}, u_{t,k}],$$

and map from reference coordinate $z_k \in [-1,1]$ by

$$r_{t,k} = a_{t,k} + h_{t,k} z_k, \qquad a_{t,k} = \frac{u_{t,k} + \ell_{t,k}}{2}, \qquad h_{t,k} = \frac{u_{t,k} - \ell_{t,k}}{2}. \tag{C1}$$

The Jacobian is constant:

$$J_t = \prod_{k=1}^{D} h_{t,k}. \tag{C2}$$

The branch stores $\ell_{t,k}, u_{t,k}, a_{t,k}, h_{t,k}$. In the same-scalar derivative these are constants and $\dot{J}_t = 0$.

21.2 Basis, Points, Weights, Ranks

Use normalized Legendre basis with the same degree p in every coordinate:

$$b_k(z_k) = \left(\sqrt{\frac{1}{2}} P_0(z_k), \sqrt{\frac{3}{2}} P_1(z_k), \ldots, \sqrt{\frac{2p-1}{2}} P_{p-1}(z_k) \right)^\top. \tag{C3}$$

Use fixed Halton-type fitting points. Let $p_k^\star$ be the k-th prime. If

$$j = \sum_{m=0}^{M} d_m (p_k^\star)^m, \qquad d_m \in \{0, \ldots, p_k^\star - 1\},$$

define

$$\mathrm{radinv}_{p_k^\star}(j) = \sum_{m=0}^{M} d_m (p_k^\star)^{-(m+1)}, \qquad z_k^{(j)} = 2\,\mathrm{radinv}_{p_k^\star}(j) - 1. \tag{C4}$$

Use $j = 1, \ldots, N_{\mathrm{fit}}$, weights

$$W = \frac{1}{N_{\mathrm{fit}}} I, \tag{C5}$$

and fixed ranks

$$R_0 = R_D = 1, \qquad R_1, \ldots, R_{D-1} \text{ declared before fitting.} \tag{C6}$$

No pivot, point, rank, or domain choice is allowed to change during a same-scalar derivative check.

21.3 Defensive Reference, Mass, And Shift

Use the uniform reference on $[-1, 1]^D$:

$$\lambda(z) = 2^{-D}. \tag{C7}$$

Choose a defensive floor $\epsilon_\tau > 0$ and set

$$\tau_t = 2^D \epsilon_\tau. \tag{C8}$$

Then $\tau_t \lambda(z) = \epsilon_\tau$. Choose the stabilizing shift at the base parameter β_0:

$$c_t = -\max_{1 \le j \le N_{\mathrm{fit}}} \log \widetilde{q}_t(z^{(j)}; \beta_0). \tag{C9}$$

The fitted square-root target is

$$y_j(\beta) = \exp(c_t/2) \sqrt{\widetilde{q}_t(z^{(j)}; \beta)}. \tag{C10}$$

For same-branch finite differences, c_t is reused for $\beta_0 + h$ and $\beta_0 - h$. Recomputing c_t changes the scalar.

21.4 Boxed Convention: Shifted Fit, Unshifted Density

The concrete branch uses this convention everywhere downstream.

$$\boxed{\begin{aligned}
&\text{Fitted TT:} && \phi_t(z;\beta) \approx e^{c_t/2} \sqrt{\widetilde{q}_t(z;\beta)}, \\
&\text{Approximate joint density:} && \widehat{q}_t(z;\beta) = e^{-c_t} \phi_t(z;\beta)^2 + \tau_t \lambda_t(z), \\
&\text{Retained numerator:} && a_t(z_t;\beta) = \int \left[e^{-c_t} \phi_t(z_t, z_{t-1};\beta)^2 + \tau_t \lambda_t(z_t, z_{t-1}) \right] \mathrm{d}z_{t-1}, \\
&\text{Normalizer:} && \widehat{Z}_t(\beta) = \int \widehat{q}_t(z;\beta)\,\mathrm{d}z = e^{-c_t} \int \phi_t(z;\beta)^2\,\mathrm{d}z + \tau_t.
\end{aligned}} \tag{C11}$$

Every carried filter, proposition, derivative, and finite-difference check uses (C11). If an implementation instead absorbs $e^{-c_t/2}$ into the TT cores, then it must remove the explicit e^{-c_t} factors everywhere. Mixing the two conventions changes the scalar.

21.5 Initialization And Sweep Order

Initialize every core to zero except the constant channel:

$$C_k[1,0,1] = 1, \qquad k = 2, \dots, D,$$

and

$$C_1[1,0,1] = \bar{y}, \qquad \bar{y} = \frac{1}{N_{\text{fit}}} \sum_{j=1}^{N_{\text{fit}}} y_j.$$

If a rank is larger than one, all non-first rank channels start at zero. This plain initialization gives a constant initial approximation and avoids a random initialization branch.

One bidirectional sweep means update cores $1, 2, \dots, D$, then $D, D-1, \dots, 1$. Fix the number of bidirectional sweeps S. The core update is the ridge solve (A1-2d). The branch stores the sweep order, S, ρ, and the final cores.

21.6 Forward-Step Object Flow

The fixed-branch forward step is a dependency chain among mathematical objects, not executable code. The stored-object flow is

$$\begin{aligned}
\text{reference points} &\longrightarrow z^{(j)} \text{ from (C4)}, \\
\text{physical points} &\longrightarrow r^{(j)} = \Psi_t(z^{(j)}), \\
\text{target values} &\longrightarrow \widetilde{q}_{t,j} = \widehat{p}_{t-1}(x_{t-1}^{(j)}; \beta) f_\beta(x_t^{(j)} \mid x_{t-1}^{(j)}) g_\beta(y_t \mid x_t^{(j)}) J_t, \\
\text{square-root fit values} &\longrightarrow y_j = \exp(c_t/2) \sqrt{\widetilde{q}_{t,j}}, \\
\text{initial cores} &\longrightarrow \text{constant-channel initialization}, \\
\text{fixed sweep objects} &\longrightarrow L_{j,k}, R_{j,k}, A_{j,k}, N_k, d_k, C_k, \\
\text{mass objects} &\longrightarrow M_{>k}, R_t, \widehat{Z}_t, \\
\text{carried objects} &\longrightarrow a_t, \widehat{p}_t.
\end{aligned} \tag{C12}$$

The fixed sweep environments are

$$L_{j,k} = H_1(z_1^{(j)}) \cdots H_{k-1}(z_{k-1}^{(j)}),$$

$$R_{j,k} = H_{k+1}(z_{k+1}^{(j)}) \cdots H_D(z_D^{(j)}),$$

with $A_{j,k}$ defined by (A1-2b) and the core update defined by (A1-2d). After fitting, square-mass contractions use (M5)–(M6):

$$R_t = \int \phi_t(z)^2 \, \mathrm{d}z, \qquad \widehat{Z}_t = e^{-c_t} R_t + \tau_t.$$

The carried numerator is the old-state marginal

$$a_t(z_t; \beta) = \int \left[e^{-c_t} \phi_t(z_t, z_{t-1}; \beta)^2 + \tau_t \lambda_t(z_t, z_{t-1}) \right] \mathrm{d}z_{t-1},$$

and the carried filter is $a_t / \widehat{Z}_t$.

21.7 Conditional-Map Inversion Protocol

For a conditional coordinate r_k, compute

$$F_k(s \mid r_{<k}) = \int_{\ell_k}^{s} \widehat{p}(r'_k \mid r_{<k})\, \mathrm{d}r'_k.$$

To sample from the conditional, given $u \in (0,1)$, solve

$$F_k(s \mid r_{<k}) - u = 0, \qquad s \in [\ell_k, u_k].$$

Use bisection as the baseline because it only needs monotonicity. Fix a tolerance $\varepsilon_{\mathrm{root}}$ and maximum iterations N_{root}. The branch stores both. The bisection update is:

$$m = \frac{a+b}{2}, \qquad \text{if } F_k(m \mid r_{<k}) < u \text{ set } a = m, \text{ else set } b = m.$$

Stop when $b - a \le \varepsilon_{\mathrm{root}}$ or the iteration count reaches N_{root}. Raise a failure diagnostic if the final residual $|F_k(m \mid r_{<k}) - u|$ exceeds the declared tolerance.

22 Fixed-Branch TT Filtering Recursion

The fixed-branch BayesFilter variant chooses all discrete approximation objects before differentiating. It is narrower than the full Zhao–Cui adaptive code, but it is the variant for which an analytical derivative can be stated without changing the scalar under the derivative.

There is one important notational separation. In the Zhao–Cui learning algorithm, the unknown parameter α may be carried as a random variable inside the posterior density. In an HMC-style likelihood calculation, the parameter is instead an external argument that is varied by the sampler. To avoid using the same symbol for both roles, this section writes the external differentiated parameter as

$$\beta \in \mathbb{R}^{m_\beta}.$$

The fixed-branch likelihood variant conditions on a trial β and filters only the latent state. Its unnormalized update is

$$q_t(x_t, x_{t-1}; \beta) = \widehat{p}_{t-1}(x_{t-1}; \beta) f_\beta(x_t \mid x_{t-1}) g_\beta(y_t \mid x_t).$$

If one also wants to carry a learned static parameter as a random coordinate, that coordinate can be appended to the state; the derivative below is still with respect to the external β, not with respect to an integration variable.

Let $z \in [-1,1]^D$ be reference coordinates and

$$r = \Psi_t(z) = (x_t, x_{t-1}).$$

Let $J_t(z) = |\det \nabla \Psi_t(z)|$. The transformed target is

$$\widetilde{q}_t(z; \beta) = \widehat{p}_{t-1}(x_{t-1}; \beta) f_\beta(x_t \mid x_{t-1}) g_\beta(y_t \mid x_t) J_t(z). \tag{FB1}$$

The branch fixes points $z^{(j)}$, weights w_j, basis functions, ranks, and a core-construction algorithm. The fitted square-root TT is

$$\phi_t(z; \beta) = G_{t,1}(z_1; \beta) \cdots G_{t,D}(z_D; \beta). \tag{FB2}$$

The fixed-branch approximate density is

$$\widehat{q}_t(z;\beta) = e^{-c_t}\phi_t(z;\beta)^2 + \tau_t\lambda_t(z), \qquad \int \lambda_t(z)\,\mathrm{d}z = 1. \tag{FB3}$$

Its normalizer is

$$\widehat{Z}_t(\beta) = \int \widehat{q}_t(z;\beta)\,\mathrm{d}z = e^{-c_t}\int \phi_t(z;\beta)^2\,\mathrm{d}z + \tau_t. \tag{FB4}$$

This is exactly the boxed convention (C11). The next filter object is the normalized marginal

$$\widehat{p}_t(x_t;\beta) = \frac{\int \widehat{q}_t(z_t, z_{t-1};\beta)\,\mathrm{d}z_{t-1}}{\widehat{Z}_t(\beta)}\,\left|\det \nabla z_t/\nabla x_t\right|. \tag{FB5}$$

The Jacobian factor appears if the filter is evaluated in physical coordinates. If the whole recursion is implemented in reference coordinates, it is absorbed into the stored domain maps.

Proposition 1 (Fixed-branch recursion is a normalized approximate filter)**.** *Fix a time horizon T, model densities f_β, g_β, initial density $p_\beta(x_0)$, and fixed branch objects for each time step. Suppose every $\widehat{q}_t(\cdot;\beta)$ defined by (FB3) is nonnegative, integrable, and has $0 < \widehat{Z}_t(\beta) < \infty$. Define $\widehat{p}_t$ by (FB5), starting from the fixed initial approximation. Then each $\widehat{p}_t$ is a normalized density on the retained state variable x_t. Moreover, the recursion is exactly the Bayes filtering recursion applied to the declared approximate joint density $\widehat{q}_t$, not to the exact joint density.*

Proof. Nonnegativity follows from $\phi_t^2 \geq 0$, $\tau_t > 0$, and $\lambda_t \geq 0$. Since $\widehat{Z}_t = \int \widehat{q}_t$ is positive and finite, the normalized joint density

$$\widehat{p}_t^{\,\mathrm{joint}}(z) = \widehat{q}_t(z)/\widehat{Z}_t$$

integrates to one. The retained filter is the marginal of this normalized joint density:

$$\int \widehat{p}_t(z_t)\,\mathrm{d}z_t = \int \left[\int \widehat{p}_t^{\,\mathrm{joint}}(z_t, z_{t-1})\,\mathrm{d}z_{t-1}\right]\,\mathrm{d}z_t = \int \widehat{p}_t^{\,\mathrm{joint}}(z)\,\mathrm{d}z = 1.$$

The change-of-variables factor in physical coordinates is the standard Jacobian factor for density transformation, so normalization is preserved. The construction uses the exact Bayes filtering algebra with q_t replaced by the declared approximation $\widehat{q}_t$. No step asserts that $\widehat{q}_t = q_t$, so the result is a normalized approximate filtering recursion rather than an exact-posterior theorem. □

23 Integrated Fixed-Branch Gradient Expansion

The preceding sections preserved the source-order Zhao–Cui annotation and the BayesFilter fixed-branch setup. The next part expands the derivative path in small mathematical steps. It deliberately starts again from scalar normalizers before returning to tensor trains, because the key claim is that the gradient differentiates the same approximate scalar computed by the fixed branch.

The merge has one important reading convention. The Zhao–Cui annotation uses θ for a random static parameter inside the sequential posterior when the paper studies joint state and parameter learning. The fixed-branch gradient part uses β for an external parameter value at which BayesFilter evaluates an approximate likelihood. These are different roles:

$$\theta \text{ can be an integration coordinate in Zhao–Cui,} \qquad \beta \text{ is the differentiated argument of } \widehat{\ell}_T. \tag{P20-B1}$$

The fixed-branch scalar therefore does not claim to differentiate the adaptive Zhao–Cui algorithm with respect to every internal coordinate. It freezes the realized numerical branch and differentiates the scalar produced by that branch as β varies:

$$B = \{\text{domains, points, basis, ranks, shifts, sweeps, tolerances}\}, \qquad \beta \mapsto \widehat{\ell}_T(\beta; B). \tag{P20-B2}$$

This distinction is what makes the next derivation useful rather than merely formal. Without it, changing β could also change ranks, pivots, maps, or fitting points; then two nearby likelihood evaluations would not be values of one mathematical function. With it, the fitted core values still change with β, but they change through the same stored fitting equations:

$$C_{t,k}(\beta; B) = \text{output of the fixed core-fitting rule at parameter } \beta. \tag{P20-B3}$$

Thus the derivative below is neither a derivative of frozen numbers nor a derivative of an adaptive search procedure. It is the derivative of the declared fixed-branch approximation. This is the contract a finite-difference diagnostic and a later implementation must obey.

24 Fixed-Branch Gradient Motivation

The filtering algorithm produces, at each time t, an approximate normalizing constant

$$\widehat{Z}_t(\beta) > 0. \tag{P19-1}$$

The parameter β is the model parameter at which we evaluate the approximate likelihood. The total approximate log likelihood is

$$\widehat{\ell}_T(\beta) = \sum_{t=1}^{T} \log \widehat{Z}_t(\beta). \tag{P19-2}$$

Formally, $\widehat{Z}_t(\beta)$ plays the role of a partition-function-like normalizer: it is an integral of a nonnegative weight. Here $\widehat{Z}_t(\beta)$ is also an integral of a nonnegative weight:

$$\widehat{Z}_t(\beta) = \int \widehat{q}_t(z; \beta)\, \mathrm{d}z. \tag{P19-3}$$

The variable z is a numerical coordinate, usually a transformed coordinate on a fixed box. The function $\widehat{q}_t$ is not the exact Bayesian density. It is the squared tensor-train approximation used by the numerical filter.

The panel cares about the gradient because parameter inference algorithms need to know how the approximate log likelihood changes when β changes:

$$\nabla_\beta \widehat{\ell}_T(\beta). \tag{P19-4}$$

The delicate point is not the derivative of log. The delicate point is that $\widehat{Z}_t$ is produced by a complicated approximation algorithm. If the approximation algorithm changes its grid, ranks, pivots, or scaling shifts when β changes, then the two likelihood evaluations may not be evaluations of the same mathematical formula. A classical analytical derivative can only differentiate one declared formula.

That is why this note uses a fixed branch.

Fixed branch. Before differentiating, freeze the computational choices: coordinate domain, fitting points, basis functions, tensor-train ranks, sweep order, ridge parameter, defensive density, stabilizing shifts, and the rule for constructing every fitted core. Do not freeze the fitted core values themselves. Those core values are outputs of the fixed fitting equations, so they remain functions of β. Then differentiate the scalar defined by the saved choices and by the core values obtained from the same saved fitting equations.

The goal is therefore:

differentiate the scalar actually computed by the saved forward pass. (P19-5)

The goal is not:

differentiate every possible adaptive version of the algorithm. (P19-6)

25 Warmup 1: A Normalizing Constant

Start with the simplest possible object. Let

$$Z(\beta) = \int_\Omega q(z;\beta)\,\mathrm{d}z, \qquad q(z;\beta) \geq 0, \qquad 0 < Z(\beta) < \infty. \tag{P19-7}$$

The normalized density is

$$p(z;\beta) = \frac{q(z;\beta)}{Z(\beta)}. \tag{P19-8}$$

The log normalizing constant is

$$L(\beta) = \log Z(\beta). \tag{P19-9}$$

Assume that $q(z;\beta)$ is differentiable in β, and that we may move the derivative inside the integral:

$$\frac{\partial}{\partial\beta}\int_\Omega q(z;\beta)\,\mathrm{d}z = \int_\Omega \frac{\partial q(z;\beta)}{\partial\beta}\,\mathrm{d}z. \tag{P19-10}$$

A sufficient condition is that, near the parameter value of interest, the absolute derivative is bounded by an integrable function:

$$\left|\frac{\partial q(z;\beta)}{\partial\beta}\right| \leq m(z), \qquad \int_\Omega m(z)\,\mathrm{d}z < \infty. \tag{P19-11}$$

Then the derivative of the log normalizer is

$$\begin{aligned}\frac{\partial L(\beta)}{\partial\beta} &= \frac{\partial}{\partial\beta}\log Z(\beta)\\ &= \frac{1}{Z(\beta)}\frac{\partial Z(\beta)}{\partial\beta}\\ &= \frac{1}{Z(\beta)}\int_\Omega \frac{\partial q(z;\beta)}{\partial\beta}\,\mathrm{d}z.\end{aligned} \tag{P19-12}$$

This is the first idea in the whole gradient. Everything later is only a way to compute the numerator and denominator in (12) for the tensor-train approximation.

For the sequential filter, the same formula is used at every time:

$$\frac{\partial\widehat{\ell}_T(\beta)}{\partial\beta} = \sum_{t=1}^{T}\frac{1}{\widehat{Z}_t(\beta)}\frac{\partial\widehat{Z}_t(\beta)}{\partial\beta}. \tag{P19-13}$$

26 Warmup 2: A Squared Approximation

The fixed-branch squared tensor-train approximation has the form

$$\widehat{q}(z;\beta) = e^{-c}\phi(z;\beta)^2 + \tau\lambda(z). \tag{P19-14}$$

Here:

$$\begin{array}{ll} c & \text{is a frozen stabilizing shift,} \\ \tau > 0 & \text{is a frozen defensive mass,} \\ \lambda(z) \geq 0 & \text{is a frozen reference density with } \int \lambda = 1, \\ \phi(z;\beta) & \text{is the fitted square-root function.} \end{array} \tag{P19-15}$$

Because c, τ, λ are frozen, the only object in (14) that changes with β is ϕ. Differentiating (14) gives

$$\begin{aligned} \dot{\widehat{q}}(z;\beta) &= \frac{\partial}{\partial\beta}\left[e^{-c}\phi(z;\beta)^2 + \tau\lambda(z)\right] \\ &= e^{-c}\frac{\partial}{\partial\beta}\phi(z;\beta)^2 + \frac{\partial}{\partial\beta}\tau\lambda(z) \\ &= 2e^{-c}\phi(z;\beta)\dot{\phi}(z;\beta). \end{aligned} \tag{P19-16}$$

The dot means $\partial/\partial\beta$.

The corresponding normalizer is

$$\begin{aligned} \widehat{Z}(\beta) &= \int \widehat{q}(z;\beta)\,\mathrm{d}z \\ &= e^{-c}\int \phi(z;\beta)^2\,\mathrm{d}z + \tau\int \lambda(z)\,\mathrm{d}z \\ &= e^{-c}\int \phi(z;\beta)^2\,\mathrm{d}z + \tau. \end{aligned} \tag{P19-17}$$

Using (16), its derivative is

$$\begin{aligned} \dot{\widehat{Z}}(\beta) &= \int \dot{\widehat{q}}(z;\beta)\,\mathrm{d}z \\ &= 2e^{-c}\int \phi(z;\beta)\dot{\phi}(z;\beta)\,\mathrm{d}z. \end{aligned} \tag{P19-18}$$

Substitution into (12) gives

$$\frac{\partial}{\partial\beta}\log\widehat{Z}(\beta) = \frac{2e^{-c}\int \phi(z;\beta)\dot{\phi}(z;\beta)\,\mathrm{d}z}{e^{-c}\int \phi(z;\beta)^2\,\mathrm{d}z + \tau}. \tag{P19-19}$$

If c, τ, or λ changed with β, then (16) would have extra terms:

$$\dot{\widehat{q}} = -\dot{c}\,e^{-c}\phi^2 + 2e^{-c}\phi\dot{\phi} + \dot{\tau}\,\lambda + \tau\dot{\lambda}. \tag{P19-20}$$

The fixed-branch derivative does not use (20). It uses the fixed-branch formula (16). This is not a minor technicality. It is the mathematical line between differentiating one saved scalar and differentiating a changing adaptive procedure.

27 Warmup 3: Two Coordinates, Rank One

Before a full tensor train, consider a two-coordinate rank-one function:

$$\phi(z_1, z_2; \beta) = h_1(z_1; \beta) h_2(z_2; \beta). \tag{P19-21}$$

The square integral is

$$\begin{aligned} R(\beta) &= \iint \phi(z_1, z_2; \beta)^2 \, \mathrm{d}z_1 \, \mathrm{d}z_2 \\ &= \iint h_1(z_1; \beta)^2 h_2(z_2; \beta)^2 \, \mathrm{d}z_1 \, \mathrm{d}z_2 \\ &= \left[\int h_1(z_1; \beta)^2 \, \mathrm{d}z_1 \right] \left[\int h_2(z_2; \beta)^2 \, \mathrm{d}z_2 \right]. \end{aligned} \tag{P19-22}$$

Define

$$R_1(\beta) = \int h_1(z_1; \beta)^2 \, \mathrm{d}z_1, \qquad R_2(\beta) = \int h_2(z_2; \beta)^2 \, \mathrm{d}z_2. \tag{P19-23}$$

Then

$$R(\beta) = R_1(\beta) R_2(\beta). \tag{P19-24}$$

Differentiate by the product rule:

$$\dot{R}(\beta) = \dot{R}_1(\beta) R_2(\beta) + R_1(\beta) \dot{R}_2(\beta). \tag{P19-25}$$

Each one-dimensional derivative is

$$\dot{R}_1(\beta) = 2 \int h_1(z_1; \beta) \dot{h}_1(z_1; \beta) \, \mathrm{d}z_1, \tag{P19-26}$$

$$\dot{R}_2(\beta) = 2 \int h_2(z_2; \beta) \dot{h}_2(z_2; \beta) \, \mathrm{d}z_2. \tag{P19-27}$$

Combining (25)–(27),

$$\begin{aligned} \dot{R}(\beta) = 2 &\left[\int h_1 \dot{h}_1 \, \mathrm{d}z_1 \right] \left[\int h_2^2 \, \mathrm{d}z_2 \right] \\ &+ 2 \left[\int h_1^2 \, \mathrm{d}z_1 \right] \left[\int h_2 \dot{h}_2 \, \mathrm{d}z_2 \right]. \end{aligned} \tag{P19-28}$$

This is the simplest version of the tensor-train mass derivative: when one factor changes, keep the other factors fixed and sum the contributions.

28 Warmup 4: Two Coordinates, Rank R

Now let

$$\phi(z_1, z_2; \beta) = \sum_{r=1}^{R} h_{1r}(z_1; \beta) h_{2r}(z_2; \beta). \tag{P19-29}$$

The square integral is

$$\begin{aligned} R(\beta) &= \iint \left[\sum_{r=1}^{R} h_{1r}(z_1) h_{2r}(z_2) \right] \left[\sum_{s=1}^{R} h_{1s}(z_1) h_{2s}(z_2) \right] \mathrm{d}z_1 \, \mathrm{d}z_2 \\ &= \sum_{r=1}^{R} \sum_{s=1}^{R} \left[\int h_{1r}(z_1) h_{1s}(z_1) \, \mathrm{d}z_1 \right] \left[\int h_{2r}(z_2) h_{2s}(z_2) \, \mathrm{d}z_2 \right]. \end{aligned} \tag{P19-30}$$

Define the two mass matrices

$$M_1[r,s] = \int h_{1r}(z_1)h_{1s}(z_1)\,\mathrm{d}z_1, \tag{P19-31}$$

$$M_2[r,s] = \int h_{2r}(z_2)h_{2s}(z_2)\,\mathrm{d}z_2. \tag{P19-32}$$

Then

$$R(\beta) = \sum_{r,s} M_1[r,s]M_2[r,s]. \tag{P19-33}$$

This is the Frobenius inner product

$$R(\beta) = \langle M_1(\beta), M_2(\beta)\rangle_F. \tag{P19-34}$$

Differentiate:

$$\dot{R} = \langle \dot{M}_1, M_2\rangle_F + \langle M_1, \dot{M}_2\rangle_F. \tag{P19-35}$$

The derivative of a mass matrix entry is

$$\dot{M}_1[r,s] = \int \dot{h}_{1r}(z_1)h_{1s}(z_1) + h_{1r}(z_1)\dot{h}_{1s}(z_1)\,\mathrm{d}z_1, \tag{P19-36}$$

and similarly for M_2. The full tensor-train mass recursion is only this calculation repeated along a chain of coordinates.

29 Warmup 5: A Fixed Linear Solve

The fitted tensor-train cores are obtained from finite-dimensional least-squares updates. One such update can be written

$$N(\beta)g(\beta) = d(\beta). \tag{P19-37}$$

Here g is the vector of unknown core coefficients in the current update, N is the regularized normal matrix, and d is the right-hand side. Differentiate both sides:

$$\frac{\partial}{\partial\beta}\{N(\beta)g(\beta)\} = \frac{\partial d(\beta)}{\partial\beta}. \tag{P19-38}$$

Use the product rule:

$$\dot{N}(\beta)g(\beta) + N(\beta)\dot{g}(\beta) = \dot{d}(\beta). \tag{P19-39}$$

Move the first term to the right:

$$N(\beta)\dot{g}(\beta) = \dot{d}(\beta) - \dot{N}(\beta)g(\beta). \tag{P19-40}$$

If $N(\beta)$ is nonsingular, then

$$\dot{g}(\beta) = N(\beta)^{-1}\left[\dot{d}(\beta) - \dot{N}(\beta)g(\beta)\right]. \tag{P19-41}$$

In computation, one should not form the inverse. One solves the linear system (40) using the same numerical linear algebra used for the forward solve.

The ridge term is important because the normal matrix has the form

$$N = A^\top W A + \rho I, \qquad \rho > 0. \tag{P19-42}$$

If W is positive semidefinite, then

$$v^\top N v = v^\top A^\top W A v + \rho v^\top v \geq \rho\|v\|^2 \tag{P19-43}$$

for all v. Hence N is positive definite and nonsingular. Without nonsingularity, the derivative of the solve may not be well defined.

30 The Fixed-Branch Forward Pass

At time t, the fixed branch constructs a nonnegative approximation

$$\widehat{q}_t(z;\beta) = e^{-c_t}\phi_t(z;\beta)^2 + \tau_t\lambda_t(z) \tag{P19-44}$$

and its normalizer

$$\widehat{Z}_t(\beta) = \int \widehat{q}_t(z;\beta)\,\mathrm{d}z. \tag{P19-45}$$

The branch is the collection of structural choices that define the scalar as a function of β. These choices include the domain, fitting points, basis, ranks, sweep order, shift, defensive term, and ridge parameter. The branch does not mean that the fitted numerical core values are held constant when β changes. Instead,

$$C_{t,k}(\beta) = \text{the core obtained by the saved fitting rule at parameter } \beta. \tag{P19-45a}$$

The derivative pass computes $\dot{C}_{t,k}(\beta)$. This distinction matters: freezing the core values would usually make the normalizer nearly constant and would not assess the derivative of the fitted approximation.

Forward object	What is stored	Why it is needed later
Domain map Ψ_t	Coordinate intervals and Jacobian	Evaluates the same transformed target at every derivative point
Fitting points $z^{(j)}$	Deterministic point array and weights	Defines the least-squares equations
Basis b_k	Basis family, degree, mass matrices	Evaluates cores and mass contractions
Ranks R_k	Fixed rank vector	Fixes core dimensions
Sweep order	Core update order and number of sweeps	Defines a finite composition of solve maps
Shift c_t	One frozen scalar	Defines the same scaled square-root target
Defensive term	τ_t, λ_t	Ensures nonnegative density floor
Cores $C_{t,k}(\beta)$	Parameter-dependent fitted coefficient arrays	Evaluate ϕ_t and $\widehat{q}_t$
Mass contractions	$M_{t,>k}(\beta)$, $M_{t,<k}(\beta)$	Compute integrals and marginals
Normalizer	$\widehat{Z}_t$	Adds $\log \widehat{Z}_t$ to the scalar
Carried filter	Numerator a_t and normalized $\widehat{p}_t$	Feeds the next time step

The transformed target fitted at time t is

$$\widetilde{q}_t(z;\beta) = \widehat{p}_{t-1}(x_{t-1}(z);\beta) f_\beta(x_t(z) \mid x_{t-1}(z)) g_\beta(y_t \mid x_t(z)) J_t(z). \tag{P19-46}$$

The fitted square-root values are

$$y_j(\beta) = e^{c_t/2}\sqrt{\widetilde{q}_t(z^{(j)};\beta)}. \tag{P19-47}$$

The tensor train is

$$\phi_t(z;\beta) = H_{t,1}(z_1;\beta)H_{t,2}(z_2;\beta)\cdots H_{t,D}(z_D;\beta), \tag{P19-48}$$

where $H_{t,k}(z_k) \in \mathbb{R}^{R_{k-1}\times R_k}$ and $R_0 = R_D = 1$.

The retained numerator for the next filter is

$$a_t(z_t;\beta) = \int \widehat{q}_t(z_t, z_{t-1};\beta)\,\mathrm{d}z_{t-1}. \tag{P19-49}$$

The retained normalized filter is

$$\widehat{p}_t(z_t;\beta) = \frac{a_t(z_t;\beta)}{\widehat{Z}_t(\beta)}. \tag{P19-50}$$

If the filter is reported in physical coordinates, multiply by the appropriate change-of-variables factor. The derivative logic is unchanged if the domain map is frozen.

31 The Fixed-Branch Derivative Pass

The derivative pass computes the derivative of each forward object that depends on β. It does not create a new branch.

Derivative object	Forward object differentiated	Formula source
$\dot{\widetilde{q}}_{t,j}$	target value at fitting point	product rule
$\dot{y}_j$	square-root fit value	chain rule
$\dot{A}_k, \dot{N}_k, \dot{d}_k$	least-squares system	environment derivatives
$\dot{C}_{t,k}$	fitted core	fixed linear solve
$\dot{M}_{t,>k}$	mass contraction	product rule in core indices
$\dot{\widehat{Z}}_t$	normalizer	squared-integral derivative
$\dot{a}_t$	carried numerator	marginal contraction derivative
$\dot{\widehat{p}}_t$	normalized carried filter	quotient rule
$\dot{\widetilde{q}}_{t+1}$	next target	product rule using $\dot{\widehat{p}}_t$

31.1 Target And Square-Root Target

Differentiate (46). At fitting point $z^{(j)}$, abbreviate

$$\widehat{p}_{t-1,j} = \widehat{p}_{t-1}(x_{t-1}^{(j)};\beta), \quad f_j = f_\beta(x_t^{(j)} \mid x_{t-1}^{(j)}), \quad g_j = g_\beta(y_t \mid x_t^{(j)}). \tag{P19-51}$$

With $J_{t,j}$ frozen, the derivative is

$$\dot{\widetilde{q}}_{t,j} = J_{t,j}\left[\dot{\widehat{p}}_{t-1,j} f_j g_j + \widehat{p}_{t-1,j}\dot{f}_j g_j + \widehat{p}_{t-1,j} f_j \dot{g}_j\right]. \tag{P19-52}$$

If the domain map depended on β, an additional $\widehat{p} f g\,\dot{J}$ term would appear. That is outside the fixed branch used here.

From (47),

$$\dot{y}_j = e^{c_t/2}\frac{1}{2\sqrt{\widetilde{q}_{t,j}}}\dot{\widetilde{q}}_{t,j}, \tag{P19-53}$$

provided $\widetilde{q}_{t,j} > 0$. A numerical implementation should use a positive floor if the target is close to zero. If such a floor is used, the flooring rule becomes part of the declared scalar. For example,

$$y_j(\beta) = e^{c_t/2}\sqrt{\max\{\widetilde{q}_{t,j}(\beta), \varepsilon_{\text{floor}}\}} \tag{P19-53a}$$

has derivative zero through the floor whenever $\widetilde{q}_{t,j} < \varepsilon_{\text{floor}}$. The formulas below describe the smooth positive-target case; a floored implementation must differentiate the floored scalar it actually evaluates.

31.2 Design Matrix And Core Solve

The design row is the first place where tensor-train notation can look more mysterious than it is. Fix one sample point $z^{(j)}$ and one core k. All cores except core k are treated as the known left and right environments for this local least-squares update. Write

$$L_{j,k}[a] = \Big[H_1(z_1^{(j)}) \cdots H_{k-1}(z_{k-1}^{(j)})\Big]_{1a}, \tag{P19-53b}$$

$$R_{j,k}[b] = \Big[H_{k+1}(z_{k+1}^{(j)}) \cdots H_D(z_D^{(j)})\Big]_{b1}. \tag{P19-53c}$$

The local core evaluated at $z_k^{(j)}$ is

$$H_k(z_k^{(j)})[a,b] = \sum_{\ell=1}^{m_k} C_k[a,\ell,b]\, b_k(z_k^{(j)})[\ell], \tag{P19-53d}$$

where m_k is the number of one-dimensional basis functions in coordinate k. The value of the tensor train at this one point is therefore

$$\begin{aligned}\phi_t(z^{(j)};\beta) &= \sum_{a=1}^{R_{k-1}}\sum_{b=1}^{R_k} L_{j,k}[a]\, H_k(z_k^{(j)})[a,b]\, R_{j,k}[b] \\ &= \sum_{a=1}^{R_{k-1}}\sum_{\ell=1}^{m_k}\sum_{b=1}^{R_k} L_{j,k}[a]\, b_k(z_k^{(j)})[\ell]\, R_{j,k}[b]\, C_k[a,\ell,b].\end{aligned} \tag{P19-53e}$$

Equation (53e) is the whole reason the local fit is a linear least-squares problem. At the fixed sample point, the unknown numbers are the entries of the core C_k. Every coefficient multiplying $C_k[a,\ell,b]$ is already known from the branch and from the other cores.

If $g_k = \operatorname{vec}(C_k)$ stacks entries in the order (a,ℓ,b), the row coefficient of $g_k[a,\ell,b]$ is

$$A_{j,k}[a,\ell,b] = L_{j,k}[a]\, b_k(z_k^{(j)})[\ell]\, R_{j,k}[b]. \tag{P19-53f}$$

The compact Kronecker expression in (57) is just (53f) written without three sums:

$$A_{j,k} = R_{j,k}^{\top} \otimes b_k(z_k^{(j)})^{\top} \otimes L_{j,k}. \tag{P19-53g}$$

To differentiate the row, keep the basis value fixed and differentiate the two environments:

$$\begin{aligned}\dot{A}_{j,k}[a,\ell,b] = {}& \dot{L}_{j,k}[a]\, b_k(z_k^{(j)})[\ell]\, R_{j,k}[b] \\ &+ L_{j,k}[a]\, b_k(z_k^{(j)})[\ell]\, \dot{R}_{j,k}[b].\end{aligned} \tag{P19-53h}$$

Equation (58) is (53h) written in the same Kronecker shorthand. If the basis or fitting point moved with β, there would be an additional term with $\dot{b}_k(z_k^{(j)})$. That term is absent because the branch keeps basis and fitting points fixed.

For one core k, the least-squares relation has rows

$$\phi_t(z^{(j)};\beta) = A_{j,k}(\beta)g_k(\beta), \tag{P19-54}$$

where $g_k = \mathrm{vec}(C_{t,k})$. Let

$$L_{j,k} = H_1(z_1^{(j)})\cdots H_{k-1}(z_{k-1}^{(j)}), \tag{P19-55}$$

$$R_{j,k} = H_{k+1}(z_{k+1}^{(j)})\cdots H_D(z_D^{(j)}). \tag{P19-56}$$

Then

$$A_{j,k} = R_{j,k}^\top \otimes b_k(z_k^{(j)})^\top \otimes L_{j,k}. \tag{P19-57}$$

Because basis values and fitting points are frozen,

$$\dot{A}_{j,k} = \dot{R}_{j,k}^\top \otimes b_k(z_k^{(j)})^\top \otimes L_{j,k} + R_{j,k}^\top \otimes b_k(z_k^{(j)})^\top \otimes \dot{L}_{j,k}. \tag{P19-58}$$

The environments are differentiated by product rule:

$$\dot{L}_{j,1} = 0, \qquad \dot{L}_{j,k+1} = \dot{L}_{j,k}H_k(z_k^{(j)}) + L_{j,k}\dot{H}_k(z_k^{(j)}), \tag{P19-59}$$

$$\dot{R}_{j,D} = 0, \qquad \dot{R}_{j,k-1} = \dot{H}_k(z_k^{(j)})R_{j,k} + H_k(z_k^{(j)})\dot{R}_{j,k}. \tag{P19-60}$$

The local core matrix derivative is

$$\dot{H}_k(z_k^{(j)})[a,b] = \sum_\ell \dot{C}_k[a,\ell,b]\, b_k(z_k^{(j)})[\ell]. \tag{P19-61}$$

The fixed ridge solve is

$$N_k g_k = d_k, \qquad N_k = A_k^\top W A_k + \rho I, \qquad d_k = A_k^\top W y. \tag{P19-62}$$

Using (40),

$$N_k\dot{g}_k = \dot{d}_k - \dot{N}_k g_k, \tag{P19-63}$$

where

$$\dot{N}_k = \dot{A}_k^\top W A_k + A_k^\top W \dot{A}_k, \tag{P19-64}$$

$$\dot{d}_k = \dot{A}_k^\top W y + A_k^\top W \dot{y}. \tag{P19-65}$$

The derivative update is therefore another linear solve with the same left-hand matrix N_k. This is why the fixed branch is implementable: the forward pass and derivative pass use the same stored computational path.

31.3 Mass Contraction And Normalizer

The full right-mass recursion is the same idea as the rank-R warmup in (29)–(36). In the warmup, $M_1[r,s]$ stored the inner products of the first-coordinate factors and $M_2[r,s]$ stored the inner products of the second-coordinate factors. In the full tensor train, $M_{>j}[b,b']$ plays the role of the already-integrated right factor, $B_j[\ell,\ell']$ is the one-coordinate basis inner product, and the two copies of C_j are the two factors coming from squaring ϕ_t.

The right mass contraction is

$$M_{>j-1}[a,a'] = \sum_{b,b',\ell,\ell'} C_j[a,\ell,b] M_{>j}[b,b'] C_j[a',\ell',b'] B_j[\ell,\ell']. \tag{P19-66}$$

Here B_j is the basis mass matrix. It is frozen. Differentiate (66):

$$\begin{aligned} \dot{M}_{>j-1}[a,a'] = &\sum_{b,b',\ell,\ell'} \dot{C}_j[a,\ell,b] M_{>j}[b,b'] C_j[a',\ell',b'] B_j[\ell,\ell'] \\ &+ \sum_{b,b',\ell,\ell'} C_j[a,\ell,b] \dot{M}_{>j}[b,b'] C_j[a',\ell',b'] B_j[\ell,\ell'] \\ &+ \sum_{b,b',\ell,\ell'} C_j[a,\ell,b] M_{>j}[b,b'] \dot{C}_j[a',\ell',b'] B_j[\ell,\ell']. \end{aligned} \tag{P19-67}$$

Start from

$$M_{>D} = 1, \qquad \dot{M}_{>D} = 0. \tag{P19-68}$$

Then (66)–(67) move from the right end of the chain to the left end. The final square integral is

$$R_t(\beta) = \int \phi_t(z;\beta)^2 \, \mathrm{d}z = M_{>0}. \tag{P19-69}$$

Its derivative is

$$\dot{R}_t(\beta) = \dot{M}_{>0}. \tag{P19-70}$$

By (17)–(18),

$$\widehat{Z}_t(\beta) = e^{-c_t} R_t(\beta) + \tau_t, \tag{P19-71}$$

$$\dot{\widehat{Z}}_t(\beta) = e^{-c_t} \dot{R}_t(\beta). \tag{P19-72}$$

31.4 Carried Filter And Next Time Step

The carried numerator is

$$a_t(z_t;\beta) = \int \widehat{q}_t(z_t, z_{t-1};\beta) \, \mathrm{d}z_{t-1}. \tag{P19-73}$$

Its derivative is

$$\dot{a}_t(z_t;\beta) = \int \dot{\widehat{q}}_t(z_t, z_{t-1};\beta) \, \mathrm{d}z_{t-1}. \tag{P19-74}$$

For the squared part of the approximation, this marginal is another tensor contraction. To make the formula concrete, suppose the retained coordinate is the last coordinate $z_D = z_t$, and the previous-state coordinates are $z_1, \ldots, z_{D-1}$. First integrate out the previous-state coordinates by the same left-to-right mass recursion:

$$M_{<0} = 1, \qquad M_{<j}[b,b'] = \sum_{a,a',\ell,\ell'} M_{<j-1}[a,a'] \, C_j[a,\ell,b] \, C_j[a',\ell',b'] \, B_j[\ell,\ell'] \tag{P19-74a}$$

for $j = 1, \dots, D-1$. The derivative of this marginal environment is

$$\begin{aligned}\dot{M}_{<j}[b,b'] = &\sum_{a,a',\ell,\ell'} \dot{M}_{<j-1}[a,a']\, C_j[a,\ell,b]\, C_j[a',\ell',b']\, B_j[\ell,\ell'] \\ &+ \sum_{a,a',\ell,\ell'} M_{<j-1}[a,a']\, \dot{C}_j[a,\ell,b]\, C_j[a',\ell',b']\, B_j[\ell,\ell'] \\ &+ \sum_{a,a',\ell,\ell'} M_{<j-1}[a,a']\, C_j[a,\ell,b]\, \dot{C}_j[a',\ell',b']\, B_j[\ell,\ell'].\end{aligned} \tag{P19-74b}$$

Start with $\dot{M}_{<0} = 0$. After the recursion reaches $D-1$, the retained-coordinate numerator is the function

$$\begin{aligned}a_t(z_D;\beta) = e^{-c_t} &\sum_{b,b',\ell,\ell'} M_{<D-1}[b,b']\, C_D[b,\ell,1]\, C_D[b',\ell',1]\, b_D(z_D)[\ell]\, b_D(z_D)[\ell'] \\ &+ \tau_t \int \lambda_t(z_D, z_{1:D-1})\, \mathrm{d}z_{1:D-1}.\end{aligned} \tag{P19-74c}$$

Its derivative, with c_t, τ_t, λ_t and the basis fixed, is

$$\begin{aligned}\dot{a}_t(z_D;\beta) = e^{-c_t} &\sum_{b,b',\ell,\ell'} \dot{M}_{<D-1}[b,b']\, C_D[b,\ell,1]\, C_D[b',\ell',1]\, b_D(z_D)[\ell]\, b_D(z_D)[\ell'] \\ &+ e^{-c_t} \sum_{b,b',\ell,\ell'} M_{<D-1}[b,b']\, \dot{C}_D[b,\ell,1]\, C_D[b',\ell',1]\, b_D(z_D)[\ell]\, b_D(z_D)[\ell'] \\ &+ e^{-c_t} \sum_{b,b',\ell,\ell'} M_{<D-1}[b,b']\, C_D[b,\ell,1]\, \dot{C}_D[b',\ell',1]\, b_D(z_D)[\ell]\, b_D(z_D)[\ell'].\end{aligned} \tag{P19-74d}$$

Equations (74a)–(74d) are the coding recipe behind the short integral identity (74). If the retained state is not the last coordinate, reorder the coordinates or use left and right environments around the retained block; the same product-rule pattern applies.

The normalized carried filter is

$$\widehat{p}_t(z_t;\beta) = \frac{a_t(z_t;\beta)}{\widehat{Z}_t(\beta)}. \tag{P19-75}$$

Differentiate by quotient rule:

$$\dot{\widehat{p}}_t(z_t;\beta) = \frac{\dot{a}_t(z_t;\beta)}{\widehat{Z}_t(\beta)} - \frac{a_t(z_t;\beta)\dot{\widehat{Z}}_t(\beta)}{\widehat{Z}_t(\beta)^2}. \tag{P19-76}$$

This is the crucial bridge between time steps. At time $t+1$, the target contains $\widehat{p}_t$, so its derivative contains $\dot{\widehat{p}}_t$. Without saving the carried-filter derivative, the next-step gradient is incomplete.

31.5 Concrete One-Coordinate Carried-Filter Storage

The quotient formula (76) is a mathematical identity. A later implementation also needs a concrete stored object. In the minimal two-coordinate fixed branch, use the normalized Legendre basis $b_1(z) \in \mathbb{R}^p$ on the retained coordinate and store the carried numerator as a coefficient matrix:

$$Q_t[\ell,m] = e^{-c_t} \sum_{r,s=1}^{R} C_1[0,\ell,r] S_2[r,s] C_1[0,m,s] + \tau_t \mathbf{1}_{\ell=0}\mathbf{1}_{m=0}, \qquad \mathrm{shape}(Q_t) = (p,p). \tag{P22-K1}$$

The retained numerator is then the quadratic form

$$a_t(z_1;\beta) = b_1(z_1)^\top Q_t b_1(z_1). \tag{P22-K2}$$

The defensive term in (K1) gives $\tau_t/2$ because the normalized Legendre constant satisfies $b_1(z)[0] = 1/\sqrt{2}$, hence

$$\tau_t b_1(z_1)[0]^2 = \tau_t/2. \tag{P22-K3}$$

The derivative coefficient matrix is

$$\begin{aligned}\dot{Q}_t[\ell,m] &= e^{-c_t}\sum_{r,s}\dot{C}_1[0,\ell,r]S_2[r,s]C_1[0,m,s] \\ &\quad + e^{-c_t}\sum_{r,s}C_1[0,\ell,r]\dot{S}_2[r,s]C_1[0,m,s] \\ &\quad + e^{-c_t}\sum_{r,s}C_1[0,\ell,r]S_2[r,s]\dot{C}_1[0,m,s], \qquad \operatorname{shape}(\dot{Q}_t) = (p,p).\end{aligned} \tag{P22-K4}$$

The normalized carried filter is stored as

$$P_t = \frac{Q_t}{\widehat{Z}_t}, \qquad \dot{P}_t = \frac{\dot{Q}_t}{\widehat{Z}_t} - \frac{Q_t\dot{\widehat{Z}}_t}{\widehat{Z}_t^2}, \qquad \operatorname{shape}(P_t) = \operatorname{shape}(\dot{P}_t) = (p,p). \tag{P22-K5}$$

For a query vector $z^{\mathrm{query}} \in [-1,1]^M$, build

$$B^{\mathrm{query}}[j,\ell] = b_1(z_j^{\mathrm{query}})[\ell], \qquad \operatorname{shape}(B^{\mathrm{query}}) = (M,p). \tag{P22-K6}$$

The carried evaluator returns

$$\begin{aligned}\widehat{p}_t^{\mathrm{ref}}[j] &= \sum_{\ell,m} B^{\mathrm{query}}[j,\ell]P_t[\ell,m]B^{\mathrm{query}}[j,m], \\ \dot{\widehat{p}}_t^{\mathrm{ref}}[j] &= \sum_{\ell,m} B^{\mathrm{query}}[j,\ell]\dot{P}_t[\ell,m]B^{\mathrm{query}}[j,m],\end{aligned} \qquad \operatorname{shape}(\widehat{p}_t^{\mathrm{ref}}) = \operatorname{shape}(\dot{\widehat{p}}_t^{\mathrm{ref}}) = (M,). \tag{P22-K7}$$

At the next time step, the query points are the previous-state coordinate of the fitting table:

$$z_j^{\mathrm{query}} = Z_{\mathrm{fit}}[j,2], \qquad p_j = \widehat{p}_{t-1}^{\mathrm{ref}}[j], \qquad \dot{p}_j = \dot{\widehat{p}}_{t-1}^{\mathrm{ref}}[j]. \tag{P22-K8}$$

The stored object is therefore not an informal function name. It is a pair of coefficient matrices and a declared query rule. This is what makes the next-step target derivative mechanically reproducible.

32 Proposition 1: The Fixed-Branch Recursion Is A Normalized Approximate Filter

Proposition 2. *Fix a parameter value β, a time horizon T, and fixed branch objects for all times. Suppose that each approximate joint density*

$$\widehat{q}_t(z_t, z_{t-1};\beta) = e^{-c_t}\phi_t(z_t, z_{t-1};\beta)^2 + \tau_t\lambda_t(z_t, z_{t-1}) \tag{P19-77}$$

is nonnegative and integrable, and that

$$0 < \widehat{Z}_t(\beta) = \int \widehat{q}_t(z_t, z_{t-1};\beta)\,\mathrm{d}z_t\,\mathrm{d}z_{t-1} < \infty. \tag{P19-78}$$

Define

$$\widehat{p}_t(z_t;\beta) = \frac{\int \widehat{q}_t(z_t, z_{t-1};\beta)\,\mathrm{d}z_{t-1}}{\widehat{Z}_t(\beta)}. \tag{P19-79}$$

Then $\widehat{p}_t(\cdot;\beta)$ is a normalized density in z_t. The recursion is a Bayesian filtering recursion for the declared approximate joint density $\widehat{q}_t$, not a proof that $\widehat{q}_t$ equals the exact Bayesian joint density.

Proof. Nonnegativity follows immediately:

$$e^{-c_t}\phi_t^2 \geq 0, \qquad \tau_t\lambda_t \geq 0. \tag{P19-80}$$

Therefore $\widehat{q}_t \geq 0$. By assumption, $\widehat{Z}_t$ is positive and finite, so

$$\widehat{p}_t^{\,\mathrm{joint}}(z_t, z_{t-1};\beta) = \frac{\widehat{q}_t(z_t, z_{t-1};\beta)}{\widehat{Z}_t(\beta)} \tag{P19-81}$$

is a normalized joint density:

$$\begin{aligned}\iint \widehat{p}_t^{\,\mathrm{joint}}(z_t, z_{t-1};\beta)\,\mathrm{d}z_t\,\mathrm{d}z_{t-1} &= \frac{1}{\widehat{Z}_t(\beta)}\iint \widehat{q}_t(z_t, z_{t-1};\beta)\,\mathrm{d}z_t\,\mathrm{d}z_{t-1} \\ &= 1.\end{aligned} \tag{P19-82}$$

The retained filter is the marginal of this normalized joint density:

$$\widehat{p}_t(z_t;\beta) = \int \widehat{p}_t^{\,\mathrm{joint}}(z_t, z_{t-1};\beta)\,\mathrm{d}z_{t-1}. \tag{P19-83}$$

Integrating (83) over z_t,

$$\begin{aligned}\int \widehat{p}_t(z_t;\beta)\,\mathrm{d}z_t &= \int \left[\int \widehat{p}_t^{\,\mathrm{joint}}(z_t, z_{t-1};\beta)\,\mathrm{d}z_{t-1}\right]\,\mathrm{d}z_t \\ &= \iint \widehat{p}_t^{\,\mathrm{joint}}(z_t, z_{t-1};\beta)\,\mathrm{d}z_t\,\mathrm{d}z_{t-1} \\ &= 1.\end{aligned} \tag{P19-84}$$

Thus $\widehat{p}_t$ is normalized. The proof used only the declared approximation $\widehat{q}_t$. It did not use the exact Bayesian density except as motivation for constructing $\widehat{q}_t$. Hence the theorem is a normalization theorem for the approximate filtering recursion. □

33 Proposition 2: The Fixed-Branch Derivative Differentiates The Same Scalar

Lemma 2 (Fixed ridge sweeps are differentiable maps)**.** *Fix one branch: the coordinate domains, basis functions, fitting points, weights, ranks, sweep order, ridge parameter, defensive terms, and shifts are all held fixed. Consider one core update written as a ridge least-squares normal equation*

$$N_k(\beta)g_k(\beta) = d_k(\beta), \qquad N_k(\beta) = A_k(\beta)^\top W_k A_k(\beta) + \varepsilon_k I,$$

where $\varepsilon_k > 0$, W_k is fixed positive semidefinite, and $N_k(\beta)$ is nonsingular in the neighborhood being differentiated. If the entries of $A_k(\beta)$ and $d_k(\beta)$ are differentiable, then

$$\dot{g}_k(\beta) = N_k(\beta)^{-1}\left[\dot{d}_k(\beta) - \dot{N}_k(\beta)g_k(\beta)\right].$$

Thus this fixed core update is differentiable. A fixed finite sweep is a finite composition of such updates, so the fitted cores produced by the fixed sweep rule are differentiable functions of β.

Proof. Because $N_k(\beta)$ is nonsingular, the core vector is

$$g_k(\beta) = N_k(\beta)^{-1} d_k(\beta).$$

Equivalently, it satisfies $N_k g_k = d_k$. Differentiate this identity:

$$\dot{N}_k g_k + N_k \dot{g}_k = \dot{d}_k.$$

Solving for $\dot{g}_k$ gives the displayed formula. The assumptions that the branch, points, weights, ranks, ridge parameter, shifts, and sweep order are fixed ensure that no discrete choice changes while differentiating. Therefore each update is an ordinary differentiable map, and composing finitely many updates gives an ordinary differentiable fitted-core map. □

Proposition 3. *Fix the branch $B = (B_1, \dots, B_T)$. Assume:*

$$\widehat{\ell}_T(\beta; B) = \sum_{t=1}^{T} \log \widehat{Z}_t(\beta; B_t), \tag{P19-85}$$

each $\widehat{Z}_t$ is positive and finite, all model density evaluations are differentiable in β, every least-squares update uses a fixed nonsingular ridge system, and differentiation under the finite-dimensional contractions and integrals is valid. Then the derivative pass defined by (52)–(76) computes

$$\frac{\partial}{\partial \beta} \widehat{\ell}_T(\beta; B). \tag{P19-86}$$

Proof. The proof follows the computational graph of the fixed branch.

First, the transformed target (46) is a product of the previous carried filter, transition density, observation density, and frozen Jacobian. The product rule therefore gives (52). If $t = 1$, the previous filter is the initial density; for $t > 1$, its derivative is supplied by the previous carried-filter derivative (76).

Second, the fitted target values (47) are differentiable functions of $\widetilde{q}_{t,j}$. The chain rule gives (53). The stabilizing shift c_t is frozen, so no $\dot{c}_t$ term appears.

Third, each core update is a fixed linear solve. Equations (37)–(41) prove that differentiating the solve yields

$$N_k \dot{g}_k = \dot{d}_k - \dot{N}_k g_k. \tag{P19-87}$$

Equations (54)–(65) identify $N_k, d_k, \dot{N}_k, \dot{d}_k$ for the fixed least-squares update. Since the number and order of solves are fixed, a finite composition of differentiable solve maps gives differentiable fitted cores.

Fourth, the square integral $R_t = \int \phi_t^2$ is computed by the mass recursion (66). Its derivative is the product-rule recursion (67). Therefore

$$\dot{R}_t = \frac{\partial}{\partial \beta} \int \phi_t(z; \beta)^2 \, \mathrm{d}z. \tag{P19-88}$$

With frozen c_t, τ_t, λ_t, (71)–(72) give the derivative of $\widehat{Z}_t$.

Fifth, the carried numerator and carried filter are defined by (73)–(75). Their derivatives are (74) and (76), obtained by differentiating under the integral and by the quotient rule. This supplies the derivative object needed by the next time step.

Finally, since each time contribution is $\log \widehat{Z}_t(\beta; B_t)$, Warmup 1 gives

$$\frac{\partial}{\partial \beta} \log \widehat{Z}_t(\beta; B_t) = \frac{\dot{\widehat{Z}}_t(\beta; B_t)}{\widehat{Z}_t(\beta; B_t)}. \tag{P19-89}$$

Summing (89) over $t = 1, \ldots, T$ gives (86).

Every object differentiated in this proof is an object in the saved branch B. The proof never differentiates a changed rank, changed grid, changed shift, changed pivot set, or changed domain. Therefore the derivative is the derivative of the same scalar $\widehat{\ell}_T(\beta; B)$, not the derivative of an adaptive algorithm that chooses a new B when β changes. □

34 What This Does Not Prove

The result above is deliberately narrow.

It does not prove exact posterior accuracy:

$$\widehat{q}_t = q_t \quad \text{is not claimed.} \tag{P19-90}$$

It proves normalization and differentiation of the declared approximation.

It does not prove ranks stay small:

$$R_k \text{ moderate for all } k \quad \text{is a diagnostic requirement, not a theorem here.} \tag{P19-91}$$

It does not differentiate adaptive decisions:

$$\frac{\partial}{\partial \beta}\{\text{chosen ranks, pivots, domains, shifts, fitting points}\} \quad \text{is not part of the formula.} \tag{P19-92}$$

It does not prove HMC convergence:

$$\text{correct local gradient of an approximate scalar} \neq \text{posterior sampling guarantee.} \tag{P19-93}$$

These caveats make the result more honest, not weaker. The fixed-branch gradient is a precise mathematical contract. Empirical accuracy, rank stability, and sampling performance must be assessed separately.

35 Finite-Difference Diagnostic

The finite-difference diagnostic checks that the value path and derivative path agree for the same branch. Choose a base parameter β_0. Build the branch once and save the structural choices

$$\begin{aligned} B = \{&\text{domains, points, basis, ranks, sweeps, shifts,} \\ &\text{defensive terms, ridge parameters,} \\ &\text{deterministic initialization rules}\}. \end{aligned} \tag{P19-94}$$

Then evaluate the same scalar at perturbed parameters while reusing those structural choices and recomputing the parameter-dependent fitted values by the same fixed equations:

$$\widehat{\ell}_T(\beta_0 + he_i; B), \qquad \widehat{\ell}_T(\beta_0 - he_i; B). \tag{P19-95}$$

That means the domains, points, ranks, shifts, and sweep order are unchanged, but the fitted core values

$$C_{t,k}(\beta_0 + he_i; B) \quad \text{and} \quad C_{t,k}(\beta_0 - he_i; B) \tag{P19-95a}$$

are recomputed from the same saved fitting rule. They are not copied from β_0. The centered finite difference is

$$D_i(h) = \frac{\widehat{\ell}_T(\beta_0 + he_i; B) - \widehat{\ell}_T(\beta_0 - he_i; B)}{2h}. \tag{P19-96}$$

For the one-parameter minimal case, the same field is

$$D(h) = \frac{\widehat{\ell}_2(\beta_0 + h; B) - \widehat{\ell}_2(\beta_0 - h; B)}{2h}. \tag{P22-FD0a}$$

Compare it to the analytical derivative

$$G_i = \partial_i \widehat{\ell}_T(\beta_0; B). \tag{P19-97}$$

The P22-local one-parameter derivative field is

$$G = \partial_\beta \widehat{\ell}_2(\beta_0; B). \tag{P22-FD0b}$$

Record

$$e_i(h) = |D_i(h) - G_i|, \tag{P19-98}$$

and

$$e_i^{\mathrm{rel}}(h) = \frac{|D_i(h) - G_i|}{1 + |G_i|}. \tag{P19-99}$$

For the one-parameter minimal case,

$$e(h) = |D(h) - G|, \qquad e_{\mathrm{rel}}(h) = \frac{|D(h) - G|}{1 + |G|}. \tag{P22-FD0c}$$

Use a small schedule such as

$$h \in \{10^{-2}, 10^{-3}, 10^{-4}, 10^{-5}\}. \tag{P19-100}$$

The P22-local step-size field is

$$H_{\mathrm{FD}} = \{10^{-2}, 10^{-3}, 10^{-4}, 10^{-5}\}. \tag{P22-FD0d}$$

The P22-local recompute-core rule is

$$C_{t,k}(\beta_0 \pm h; B) = \text{the core produced by the fixed fitting rule at } \beta_0 \pm h, \tag{P22-FD0e}$$

not the core copied from β_0. The report should be a declared mathematical object, not an informal output list. For the one-parameter $T = 2$ minimal case, record

$$\begin{aligned} \mathcal{R}_{\mathrm{FD}} = \{&\widehat{\ell}_2(\beta_0; B),\ G,\ (h_i, D(h_i), e(h_i), e_{\mathrm{rel}}(h_i))_{i=1}^4, \\ &I_-(h_i), I_+(h_i), K_-(h_i), K_+(h_i),\ W_i,\ \text{status}\}. \end{aligned} \tag{P22-FD1}$$

Here

$$I_\pm(h_i) = \mathbf{1}\{B(\beta_0 \pm h_i) = B\} \tag{P22-FD2}$$

checks branch-manifest equality, and

$$K_\pm(h_i) = \mathbf{1}\{C_k(\beta_0 \pm h_i; B) \text{ was recomputed by the fixed fitting rule}\} \tag{P22-FD3}$$

checks that perturbed cores were not copied from β_0. A decreasing error window is recorded by

$$W_i = \mathbf{1}\{e(h_i) > e(h_{i+1}) > e(h_{i+2})\}, \qquad i = 1, 2. \tag{P22-FD4}$$

The status rule is

$$\text{status} = \begin{cases} \text{branch identity failure}, & \min_{i,\pm} I_{\pm}(h_i) = 0, \\ \text{copied-core failure}, & \min_{i,\pm} K_{\pm}(h_i) = 0, \\ \text{decreasing-window agreement}, & \max(W_1, W_2) = 1, \\ \text{inconclusive: no decreasing window}, & \text{otherwise.} \end{cases} \tag{P22-FD5}$$

The status is a derivative-consistency diagnostic for the fixed branch. It is not a posterior-accuracy theorem, a rank-stability theorem, or an HMC convergence result.

$$\begin{aligned} \mathcal{N}_{\mathrm{FD}} = \{&\text{not exact posterior accuracy}, \\ &\text{not rank-stability certification}, \\ &\text{not HMC convergence certification}\}. \end{aligned} \tag{P22-FD7}$$

A healthy derivative check typically shows decreasing error over a range of h, followed by roundoff or fitting-noise limitations. If the error does not decrease on any reasonable range, the derivative path is not yet aligned with the value path.

$$\exists i \in \{1, 2\} : W_i = 1 \quad \Longleftrightarrow \quad \text{there is a decreasing finite-difference error window.} \tag{P22-FD8}$$

The local P22 branch identity condition is mandatory:

$$B(\beta_0) = B(\beta_0 + h) = B(\beta_0 - h) = B. \tag{P22-FD6}$$

If the implementation rebuilds ranks, points, shifts, or domains at the perturbed values, then the centered difference is no longer assessing the derivative of $\widehat{\ell}_2(\beta; B)$. It is comparing a different adaptive calculation. If the implementation copies the numerical cores from β_0 without resolving the fixed fitting equations at the perturbed parameter, then it is also assessing the wrong object. The correct diagnostic freezes the branch choices but allows all differentiable outputs of those choices to change with β.

$$\begin{aligned} \min_{i,\pm} I_{\pm}(h_i) = 0 \quad &\Rightarrow \quad \text{the compared values use different branches,} \\ \min_{i,\pm} K_{\pm}(h_i) = 0 \quad &\Rightarrow \quad \text{the perturbed values copied old numerical cores,} \\ \max(W_1, W_2) = 0 \quad &\Rightarrow \quad \text{the diagnostic is inconclusive for derivative agreement.} \end{aligned} \tag{P22-FD9}$$

36 Minimal Mathematical Example

The smallest useful mathematical example should be nonlinear but one-dimensional:

$$x_t = \beta x_{t-1} + \sigma \epsilon_t, \qquad \epsilon_t \sim N(0, 1), \tag{P19-102}$$

$$y_t = \sin(x_t) + \eta_t, \qquad \eta_t \sim N(0, r^2). \tag{P19-103}$$

Use $T = 2$. The first time step defines

$$q_1(x_1, x_0; \beta) = p_\beta(x_0) f_\beta(x_1 \mid x_0) g_\beta(y_1 \mid x_1). \tag{P19-104}$$

It fits ϕ_1, computes

$$\widehat{Z}_1(\beta) = e^{-c_1} \int \phi_1(z_1, z_0; \beta)^2 \, \mathrm{d}z_1 \, \mathrm{d}z_0 + \tau_1, \tag{P19-105}$$

and stores

$$\widehat{p}_1(z_1; \beta) = \frac{\int \left[e^{-c_1} \phi_1(z_1, z_0; \beta)^2 + \tau_1 \lambda_1(z_1, z_0) \right] \, \mathrm{d}z_0}{\widehat{Z}_1(\beta)}. \tag{P19-106}$$

The second time step depends on that stored object:

$$q_2(x_2, x_1; \beta) = \widehat{p}_1(x_1; \beta) f_\beta(x_2 \mid x_1) g_\beta(y_2 \mid x_2). \tag{P19-107}$$

The scalar checked by the example is

$$\widehat{\ell}_2(\beta) = \log \widehat{Z}_1(\beta) + \log \widehat{Z}_2(\beta). \tag{P19-108}$$

The mathematical report object must contain

$$\widehat{\ell}_2(\beta_0), \qquad \partial_\beta \widehat{\ell}_2(\beta_0), \qquad D(h), \qquad |D(h) - \partial_\beta \widehat{\ell}_2(\beta_0)|. \tag{P19-109}$$

It must also contain the branch manifest identifier used for β_0, $\beta_0 + h$, and $\beta_0 - h$. The mathematical pass condition is that those branch identifiers are identical and the derivative error has a stable decreasing window before numerical roundoff dominates.

37 Integrated Conclusion

The fixed-branch gradient is built from five elementary ideas:

normalizer derivative	$\partial \log Z = \dot{Z}/Z$,	
squared density derivative	$\partial(\phi^2) = 2\phi\dot{\phi}$,	
mass contraction derivative	differentiate every factor once,	(P19-110)
linear solve derivative	$N\dot{g} = \dot{d} - \dot{N}g$,	
carried filter derivative	$\partial(a/Z) = \dot{a}/Z - a\dot{Z}/Z^2$.	

The tensor-train notation makes these ideas look large, but it does not change their nature. Once the branch is fixed, the forward pass defines one scalar. The derivative pass differentiates that scalar. What remains for empirical work is to assess whether the approximation is accurate enough, stable enough, and useful enough for the target scientific model.

38 Reader Orientation Blocks For The Fixed-Branch Derivation

This section collects neutral orientation blocks for the fixed-branch part. They are not review gates. They simply state, after the derivation, what has been established and what a later implementation must preserve.

Squared-density block

established	$\widehat{q}_t = e^{-c_t}\phi_t^2 + \tau_t\lambda_t \geq 0,$
stored	ϕ_t through $C_{t,k}$ and $\dot{\phi}_t$ through $\dot{C}_{t,k},$
integrated	$e^{-c_t}\phi_t(z)^2 + \tau_t\lambda_t(z),$
differentiated	$e^{-c_t}\phi_t(z)^2 \mapsto 2e^{-c_t}\phi_t(z)\dot{\phi}_t(z),$
fixed	$c_t, \tau_t, \lambda_t,$ basis, domain, ranks, points,
failure mode	differentiating a different fitted square root than the one squared.

(P22-R1)

Mass-contraction block

established	$R_t = \int \phi_t(z)^2 \, \mathrm{d}z$ is computed by TT mass contractions,
stored	$C_{t,k}, \dot{C}_{t,k}, B_{t,k}, M_{>k}, \dot{M}_{>k},$
integrated	ϕ_t^2 one coordinate at a time,
differentiated	each product factor once, then summed,
fixed	$B_{t,k},$ coordinate order, rank pattern, basis family,
failure mode	omitting a product-rule term in $\dot{M}_{>k}$.

(P22-R2)

Fixed-solve block

established	$N_k g_k = d_k \mapsto N_k\dot{g}_k = \dot{d}_k - \dot{N}_k g_k,$
stored	A_k, N_k, d_k, g_k and dotted counterparts,
integrated	nothing; this is a finite-dimensional fitting solve,
differentiated	the fixed normal equations,
fixed	$W, \rho, Z_{\mathrm{fit}}, B_{\mathrm{val}},$ sweep order,
failure mode	changing the solve system or copying $g_k(\beta_0)$ at perturbed parameters.

(P22-R3)

Carried-filter block

established	$\widehat{p}_t = a_t/\widehat{Z}_t$ and $\dot{\widehat{p}}_t = \dot{a}_t/\widehat{Z}_t - a_t\dot{\widehat{Z}}_t/\widehat{Z}_t^2,$
stored	$Q_t, \dot{Q}_t, P_t, \dot{P}_t$ with shape $(p, p),$
integrated	$\widehat{q}_t(z_t, z_{t-1})$ over $z_{t-1},$
differentiated	$Q_t/\widehat{Z}_t$ by the quotient rule,
fixed	basis, query coordinate, branch, defensive term,
failure mode	feeding time $t+1$ without $\dot{P}_t$ or with an undeclared query rule.

(P22-R4)

A Source Coverage Summary

The main text reconstructs the displayed mathematical spine of Zhao–Cui Sections 1–3 and 5. Equations (1)–(26) and (30)–(35), Algorithms 1–5, the notation formulas, the square-root error inequalities, the upper and lower conditional map formulas, the particle and smoothing weight normalizations, and the preconditioning composition formulas are represented in BayesFilter notation. Section 4 error propagation and Section 6 numerical examples are not reconstructed as the central object of this note; they are used only to explain what the method does and does not prove.